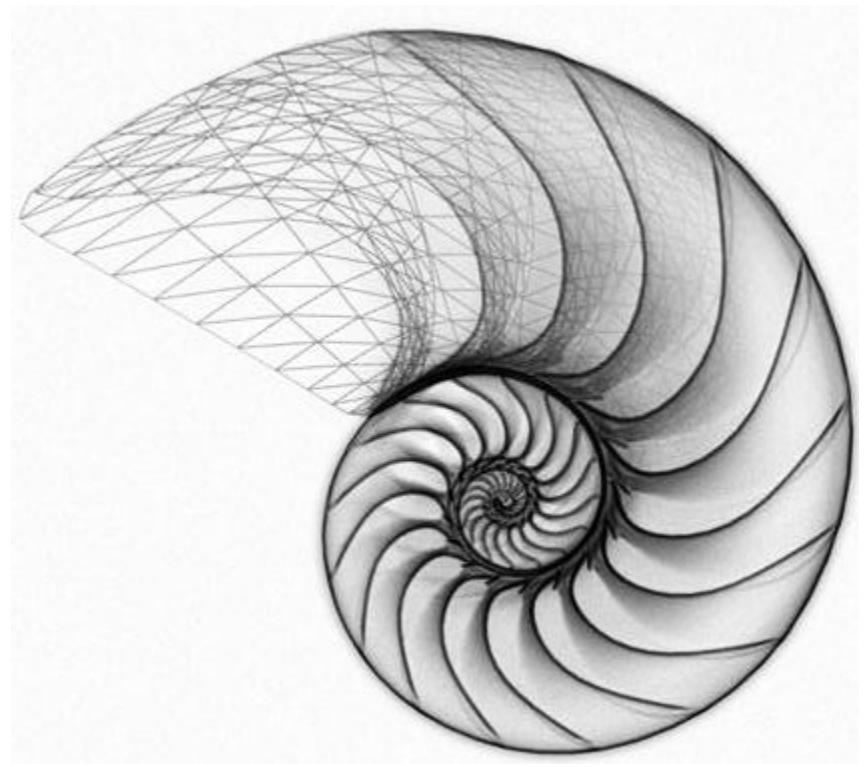




Lecture 18

Sorting

CSE225: Data Structures and Algorithms

Bubble Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Bubble Sort

5	1	3	4	6	2
---	---	---	---	---	---



Comparison

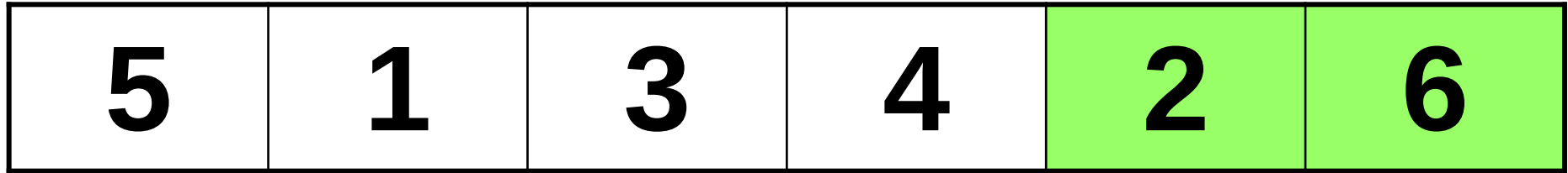


Data Movement



Sorted

Bubble Sort



Comparison



Data Movement



Sorted

Bubble Sort

5	1	3	4	2	6
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Bubble Sort



Comparison



Data Movement



Sorted

Bubble Sort

5	1	3	2	4	6
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Bubble Sort

5	1	2	3	4	6
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Bubble Sort

5	1	2	3	4	6
---	---	---	---	---	---



Comparison



Data Movement



Sorted

Bubble Sort

5	1	2	3	4	6
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Bubble Sort

1	5	2	3	4	6
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Bubble Sort

1	5	2	3	4	6
----------	----------	----------	----------	----------	----------



Comparison

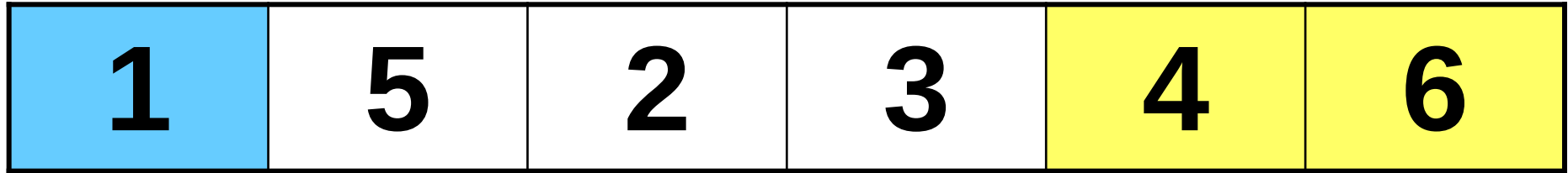


Data Movement



Sorted

Bubble Sort



Comparison

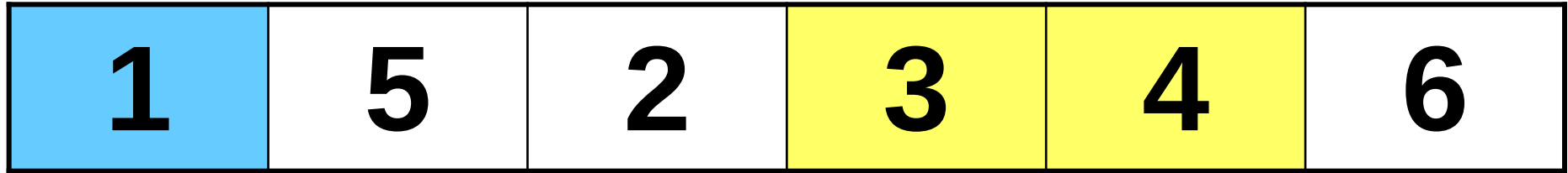


Data Movement



Sorted

Bubble Sort



Comparison

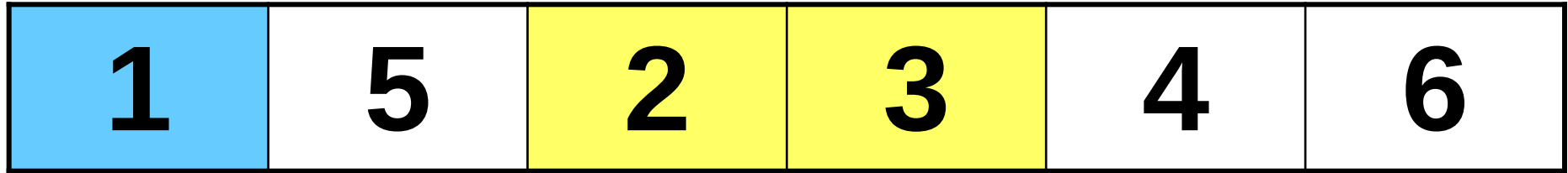


Data Movement



Sorted

Bubble Sort



Comparison

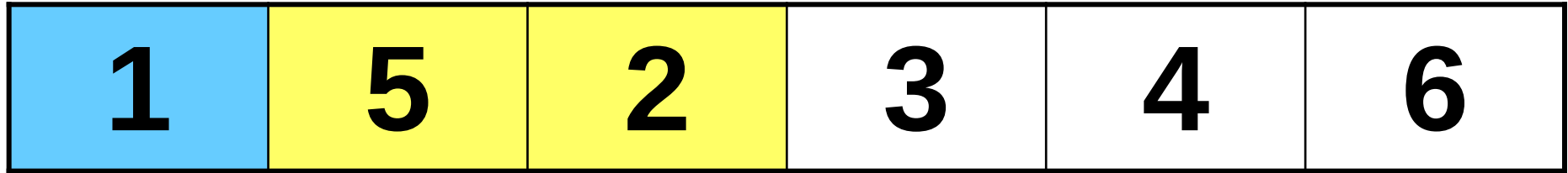


Data Movement



Sorted

Bubble Sort



Comparison

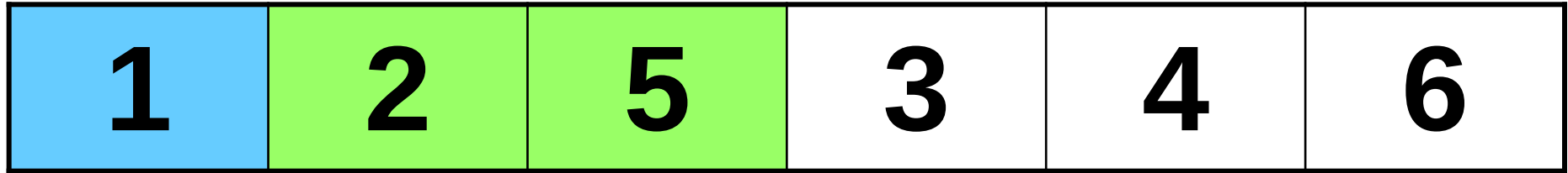


Data Movement



Sorted

Bubble Sort



Comparison

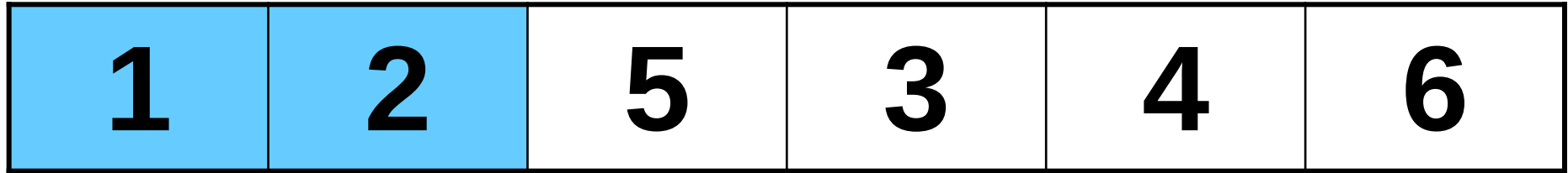


Data Movement



Sorted

Bubble Sort



Comparison

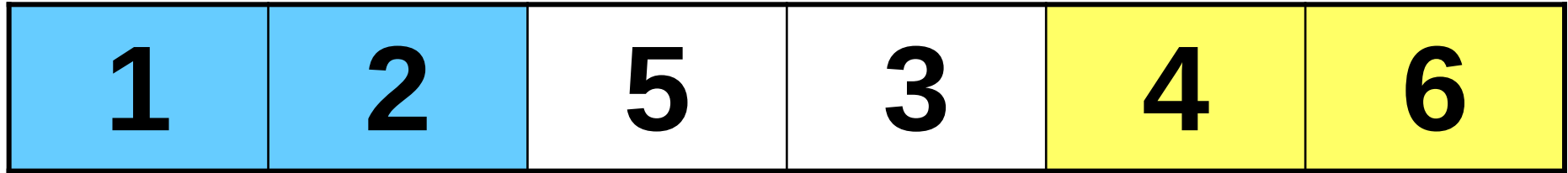


Data Movement



Sorted

Bubble Sort



Comparison

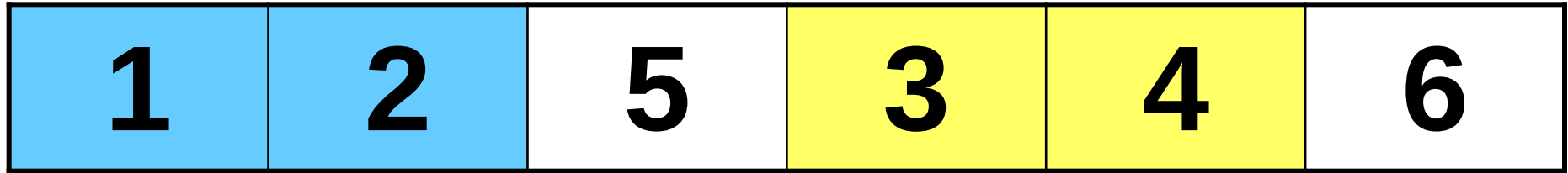


Data Movement



Sorted

Bubble Sort



Comparison

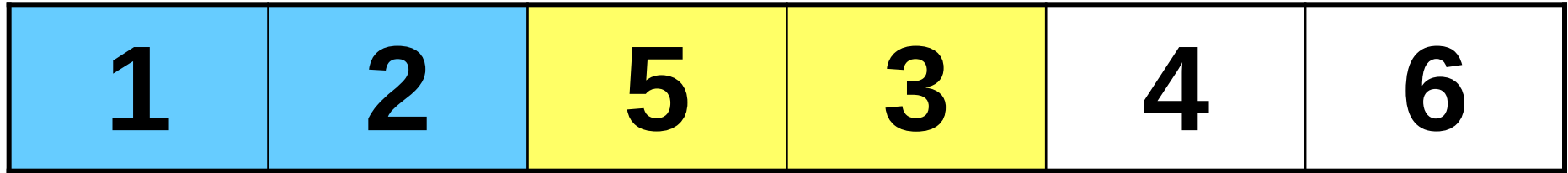


Data Movement



Sorted

Bubble Sort



Comparison

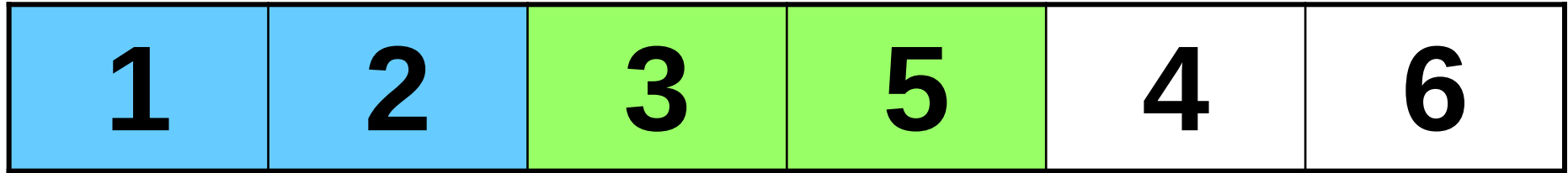


Data Movement



Sorted

Bubble Sort



Comparison

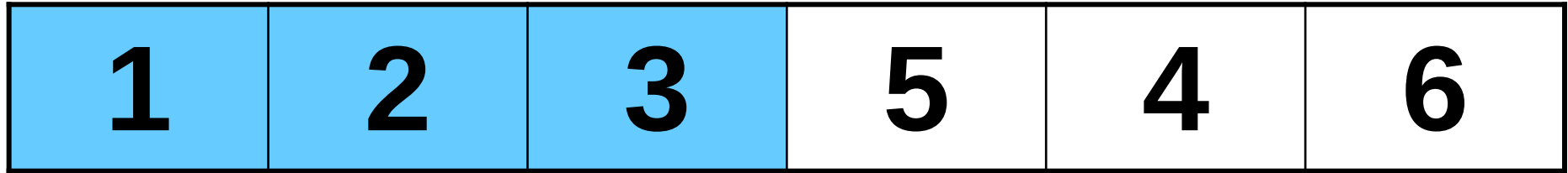


Data Movement



Sorted

Bubble Sort



Comparison

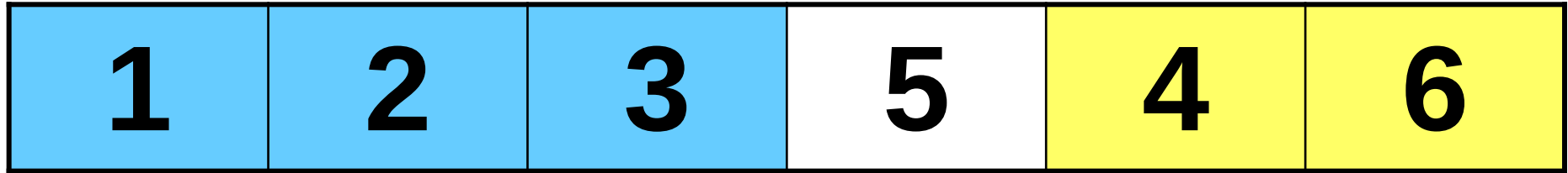


Data Movement



Sorted

Bubble Sort



Comparison

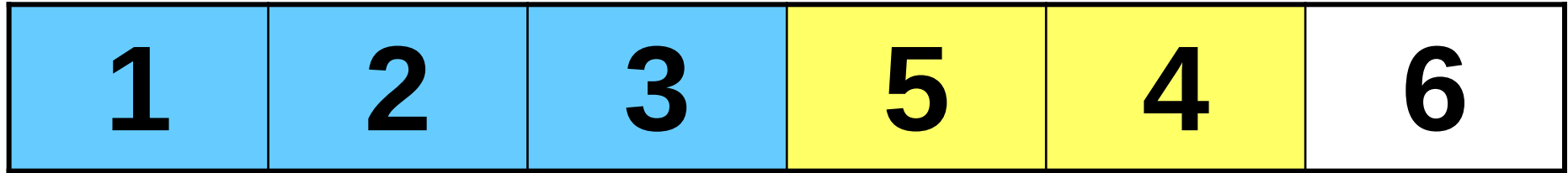


Data Movement



Sorted

Bubble Sort



Comparison

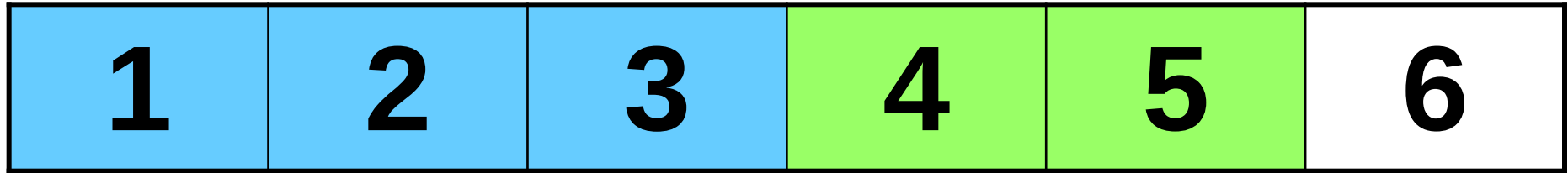


Data Movement



Sorted

Bubble Sort



Comparison

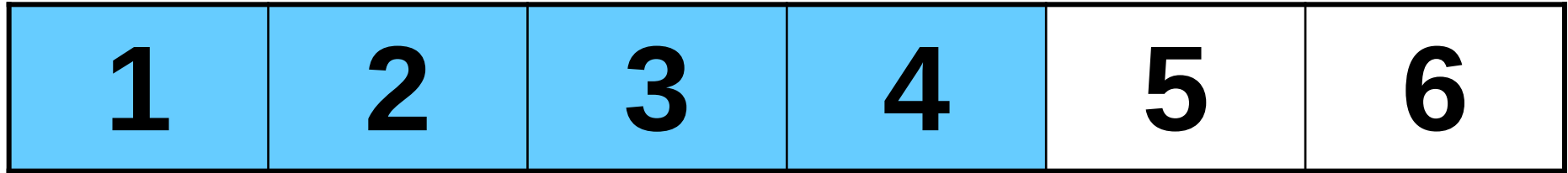


Data Movement



Sorted

Bubble Sort



Comparison

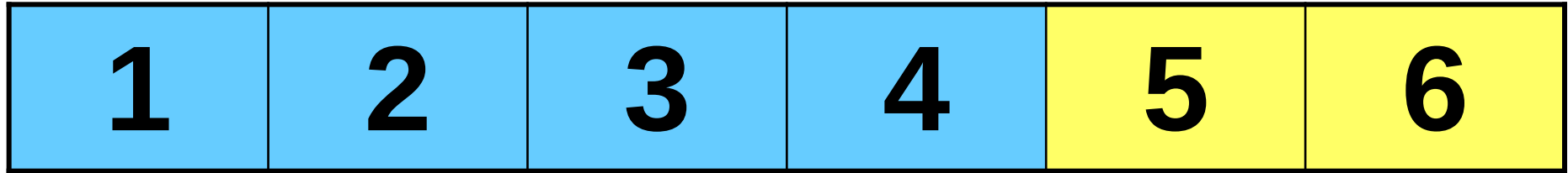


Data Movement



Sorted

Bubble Sort



Comparison

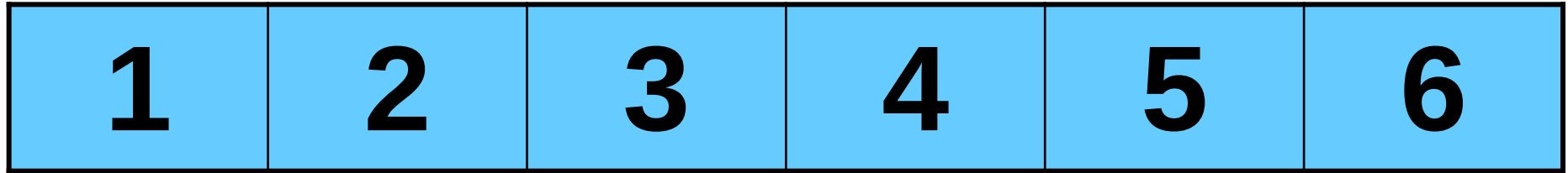


Data Movement



Sorted

Bubble Sort



Comparison



Data Movement



Sorted

Bubble Sort

```
template<class ItemType>
void Swap(ItemType& item1, ItemType& item2)
{
    ItemType tempItem;

    tempItem = item1;
    item1 = item2;
    item2 = tempItem;
}
```

Bubble Sort

```
template<class ItemType>
void BubbleUp(ItemType values[] , int startIndex, int endIndex)
{
    for (int index = endIndex; index > startIndex; index--)
        if (values[index-1] > values[index])
            Swap(values[index-1], values[index]);
}
```

```
template<class ItemType>
void BubbleSort(ItemType values[], int numValues)
{
    int current = 0;

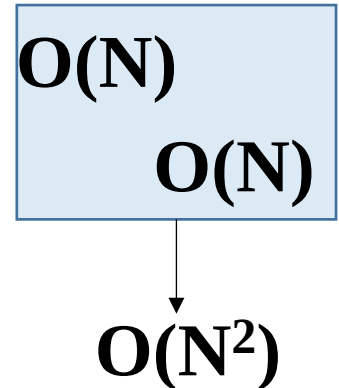
    while (current < numValues - 1)
    {
        BubbleUp(values, current, numValues-1);
        current++;
    }
}
```

Bubble Sort

```
template<class ItemType>
void BubbleUp(ItemType values[] , int startIndex, int endIndex)
{
    for (int index = endIndex; index > startIndex; index--)
        if (values[index-1] > values[index])
            Swap(values[index-1], values[index]);
}
```

```
template<class ItemType>
void BubbleSort(ItemType values[], int numValues)
{
    int current = 0;

    while (current < numValues - 1)
    {
        BubbleUp(values, current, numValues-1);
        current++;
    }
}
```



Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------

↑
Current



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------

↑
Current



Comparison

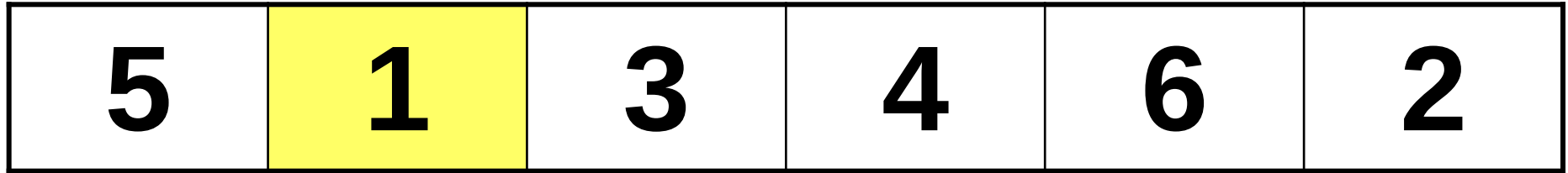


Data Movement



Sorted

Selection Sort



↑
Current



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
---	---	---	---	---	---

↑
Current



Comparison

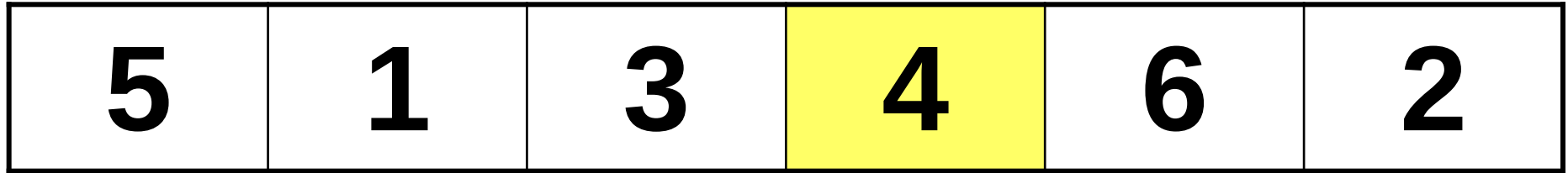


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

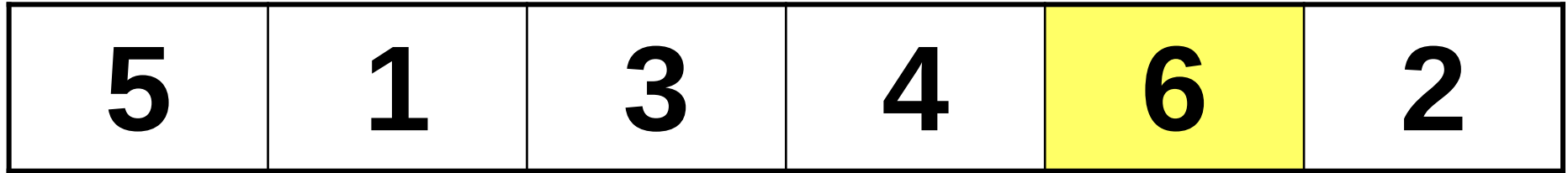


Data Movement



Sorted

Selection Sort



↑
Current



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
---	---	---	---	---	---

↑
Current



Comparison



Data Movement



Sorted

Selection Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------

↑
Current

↑
Smallest



Comparison

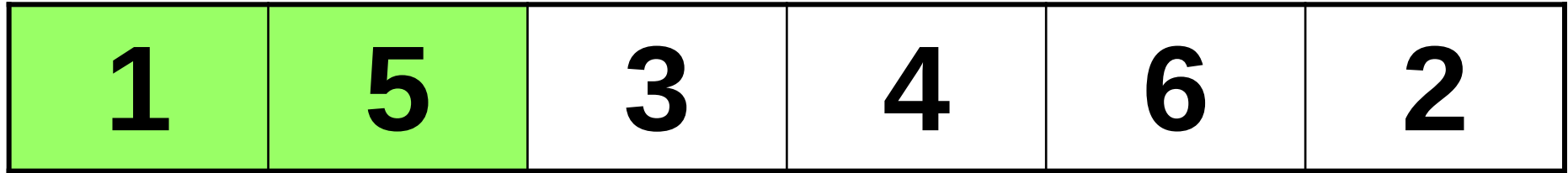


Data Movement



Sorted

Selection Sort



↑
Current

↑
Smallest



Comparison



Data Movement



Sorted

Selection Sort

1	5	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison

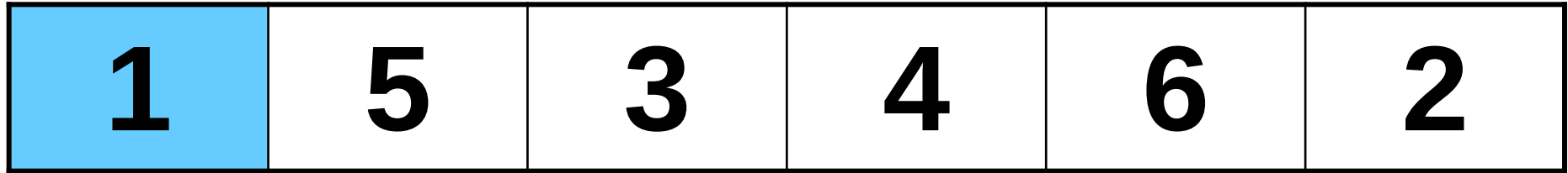


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

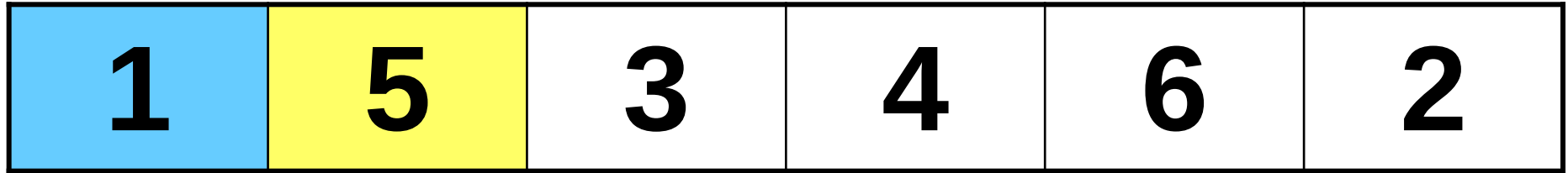


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

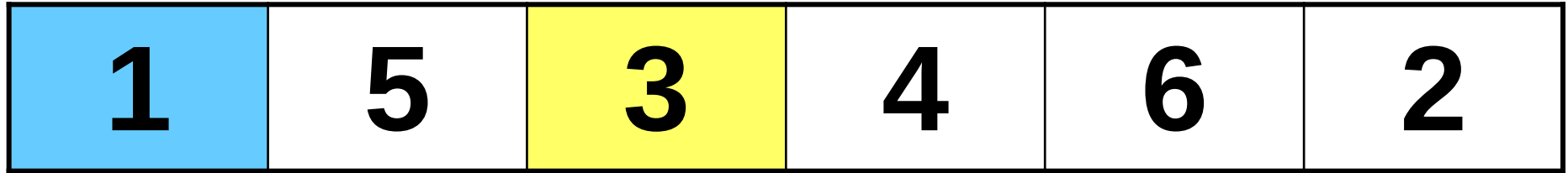


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

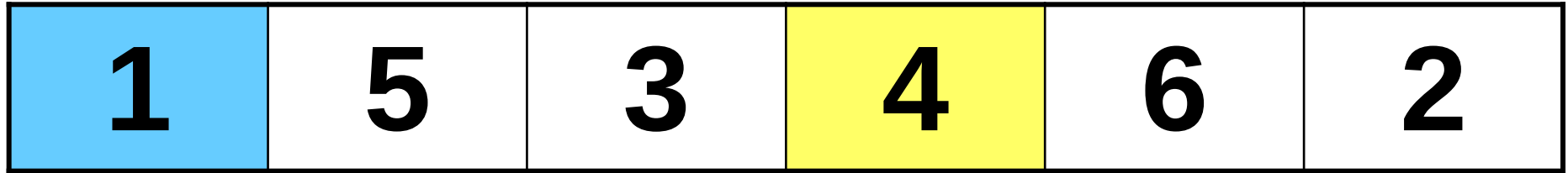


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

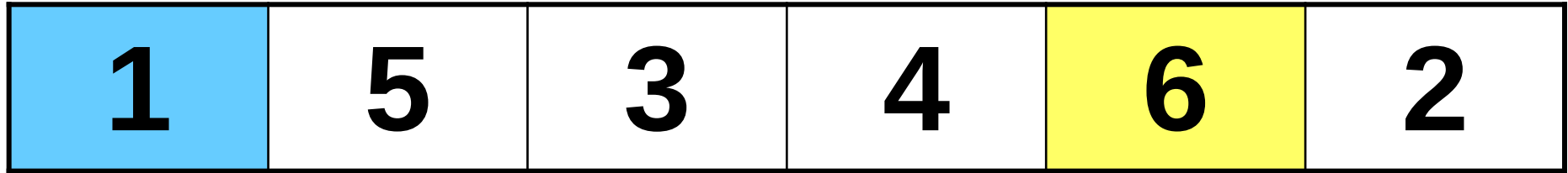


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

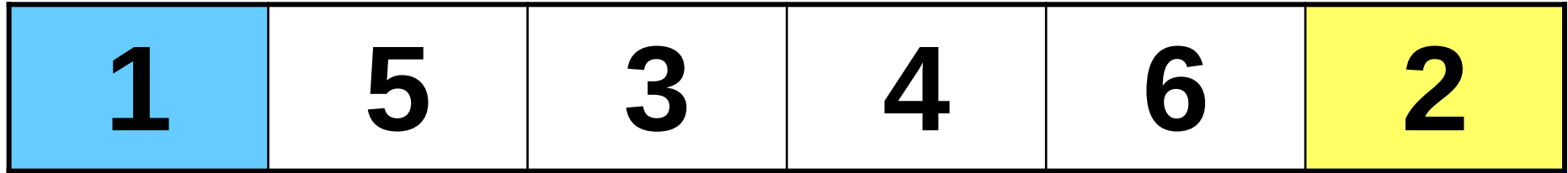


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

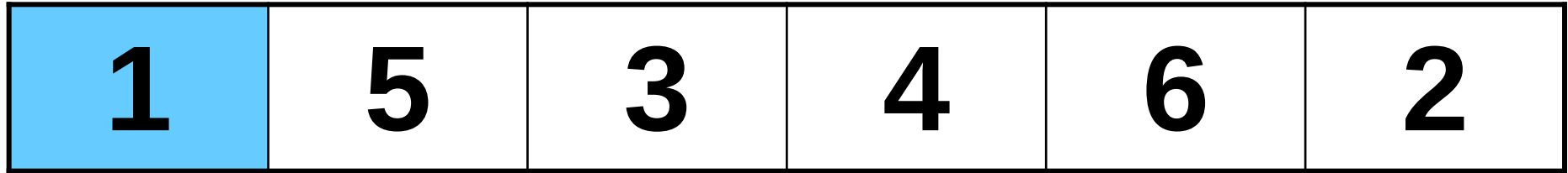


Data Movement



Sorted

Selection Sort



↑
Current

↑
Smallest



Comparison

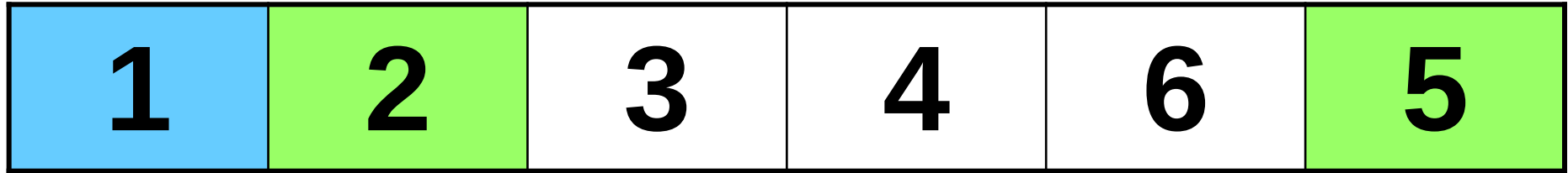


Data Movement



Sorted

Selection Sort



↑
Current

↑
Smallest



Comparison



Data Movement



Sorted

Selection Sort

1	2	3	4	6	5
----------	----------	----------	----------	----------	----------



Comparison

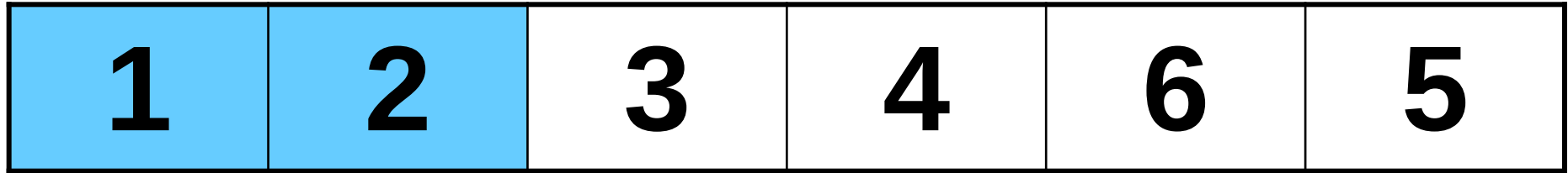


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

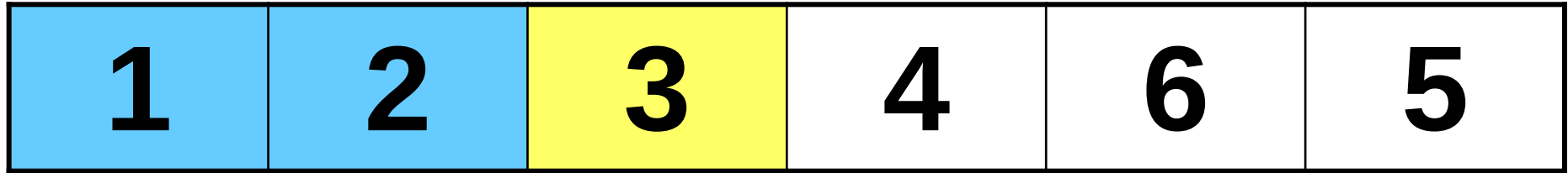


Data Movement



Sorted

Selection Sort



Comparison

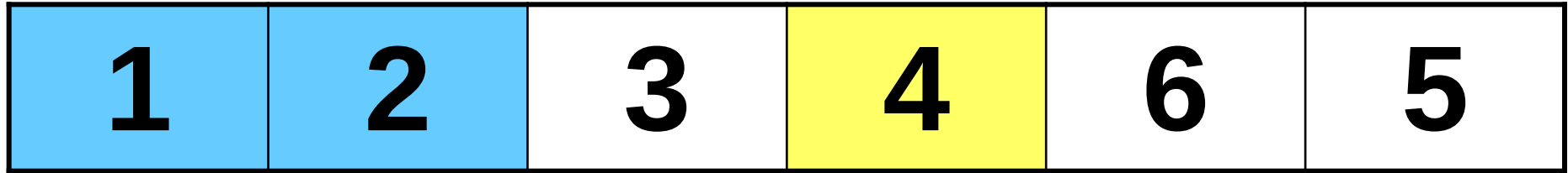


Data Movement



Sorted

Selection Sort



Comparison

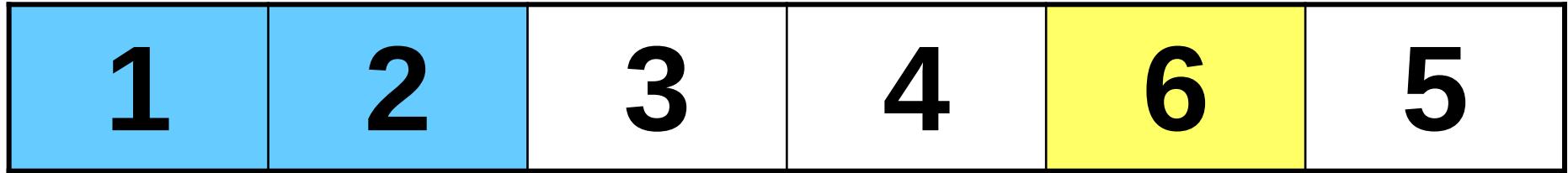


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

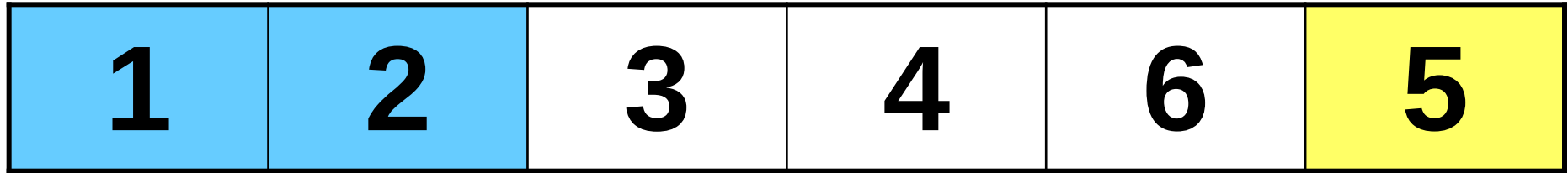


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

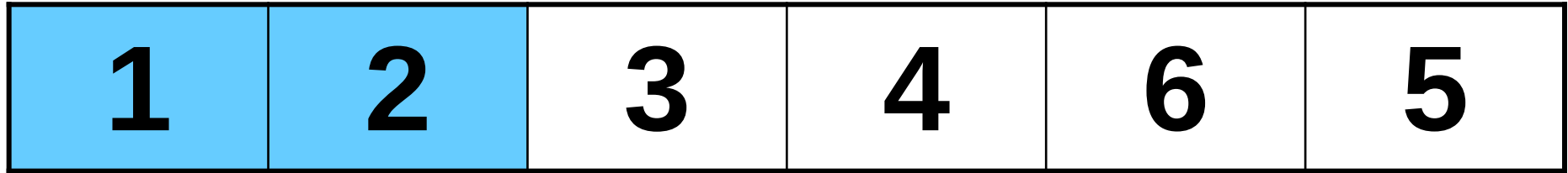


Data Movement



Sorted

Selection Sort



↑
Current
↑
Smallest



Comparison

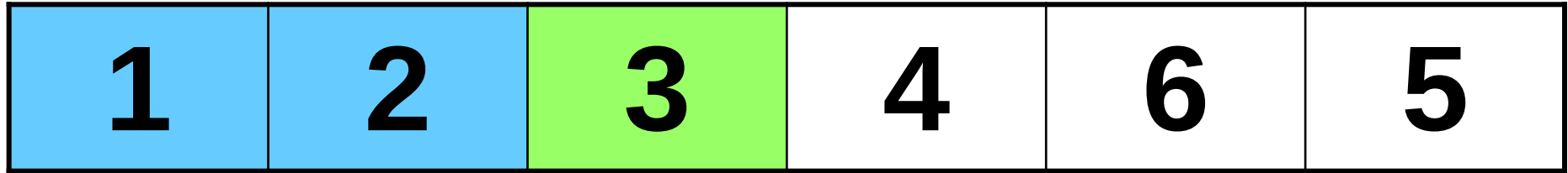


Data Movement



Sorted

Selection Sort



↑
Current
↑
Smallest



Comparison

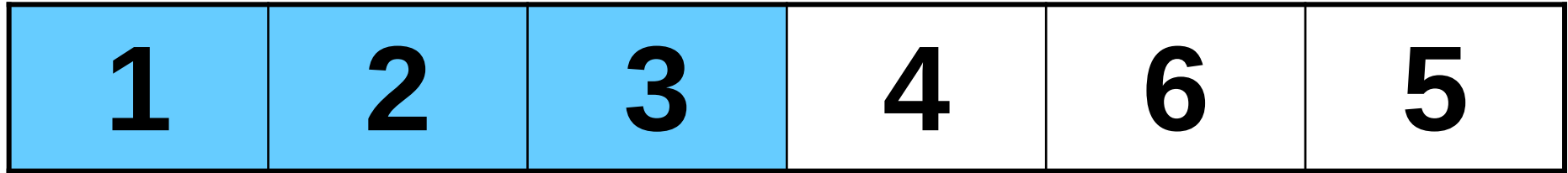


Data Movement



Sorted

Selection Sort



Comparison

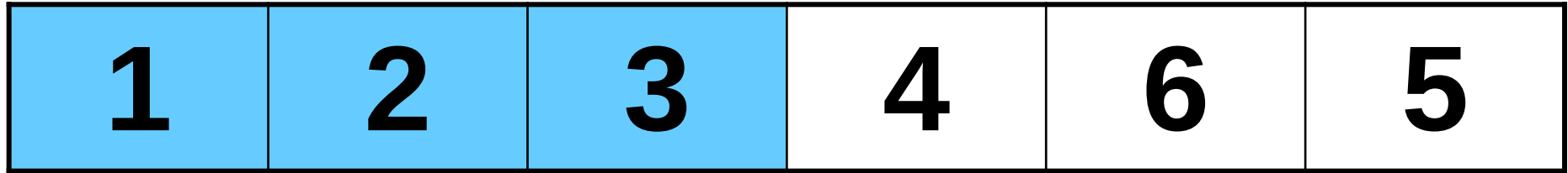


Data Movement



Sorted

Selection Sort



Comparison

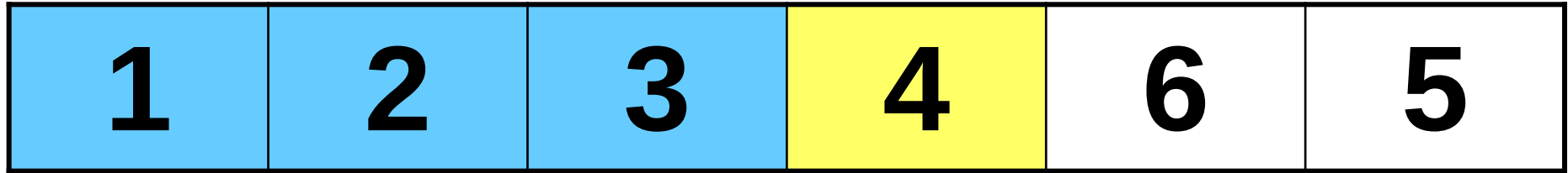


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

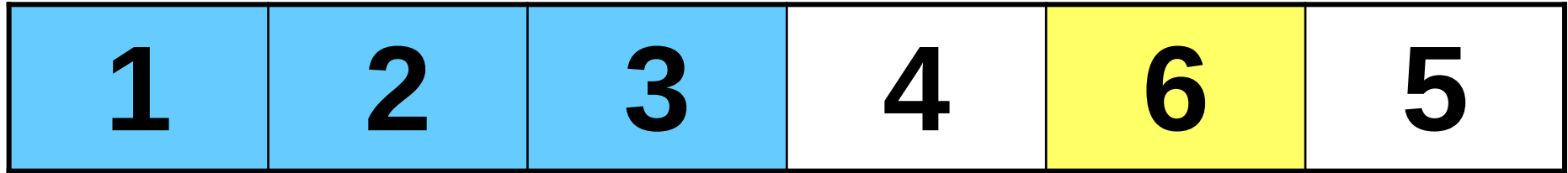


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

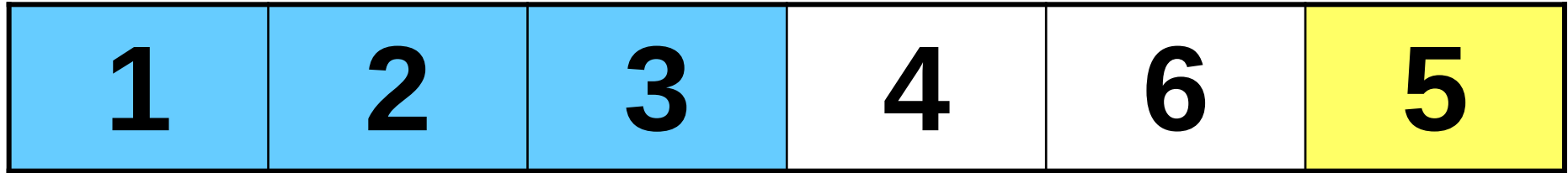


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

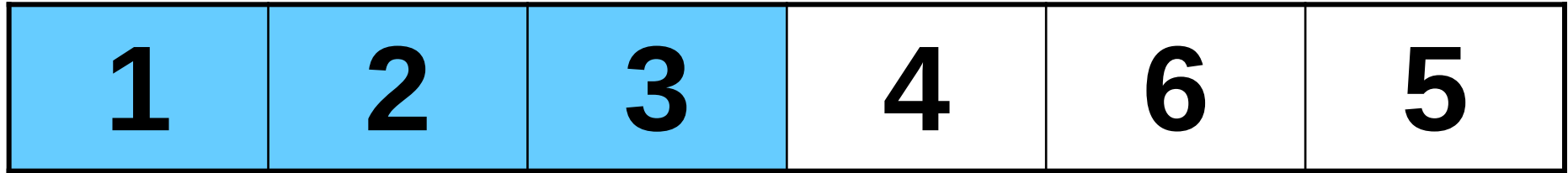


Data Movement



Sorted

Selection Sort



↑
Current
↑
Smallest



Comparison

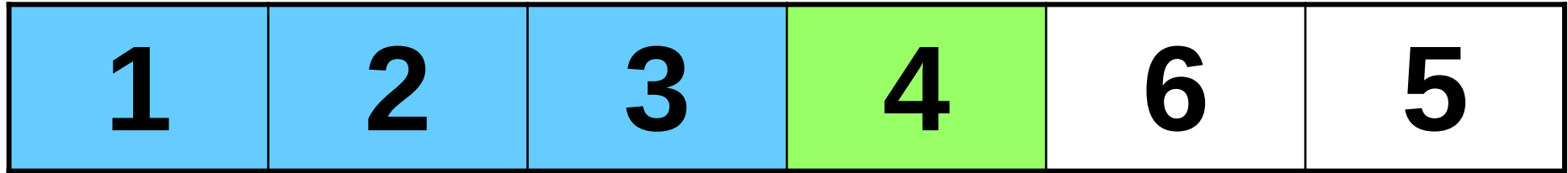


Data Movement



Sorted

Selection Sort



↑
Current
↑
Smallest



Comparison

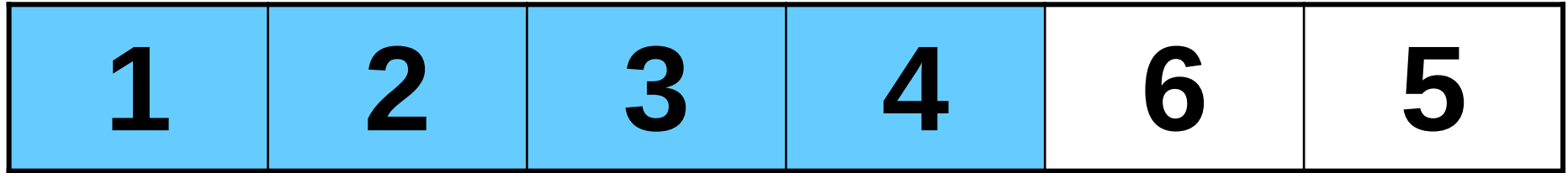


Data Movement



Sorted

Selection Sort



Comparison

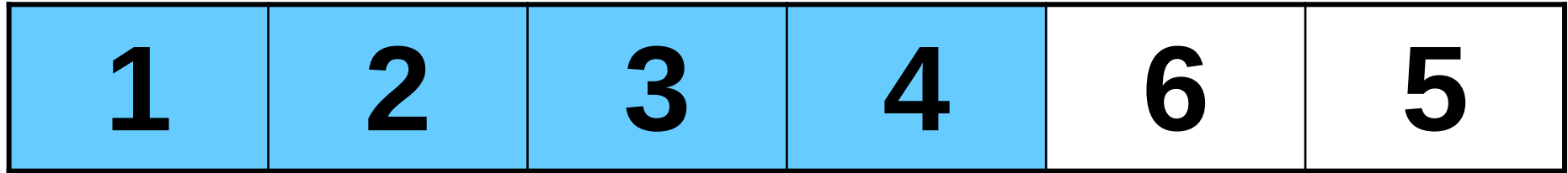


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

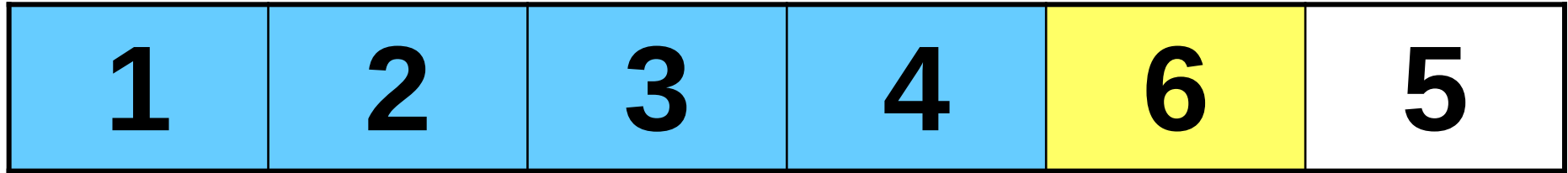


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

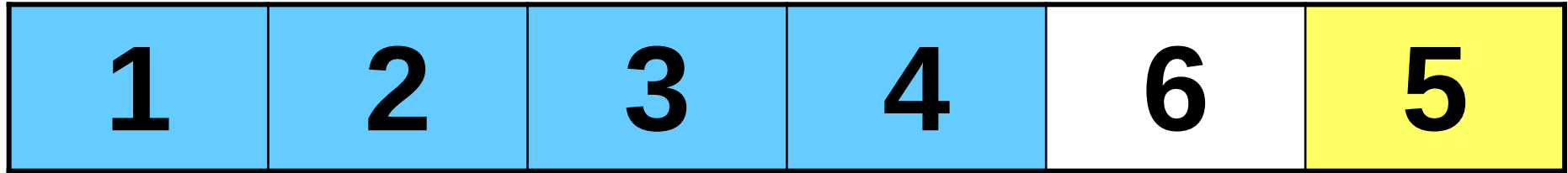


Data Movement



Sorted

Selection Sort



↑
Current



Comparison

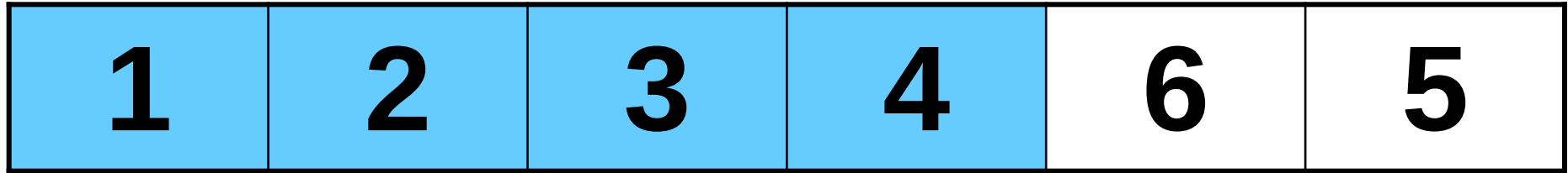


Data Movement



Sorted

Selection Sort



↑
Current

↑
Smallest



Comparison

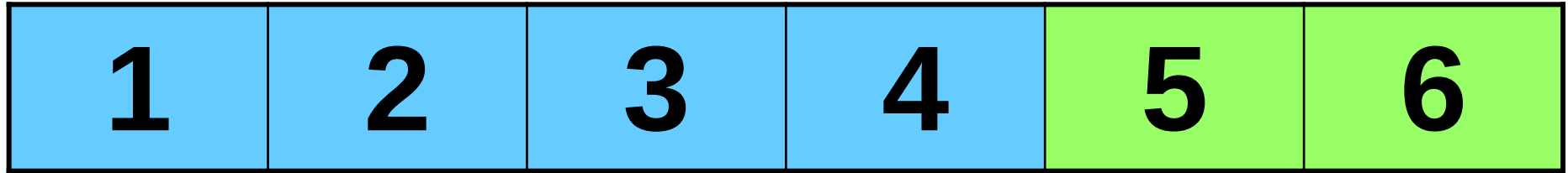


Data Movement



Sorted

Selection Sort



↑
Current

↑
Smallest



Comparison

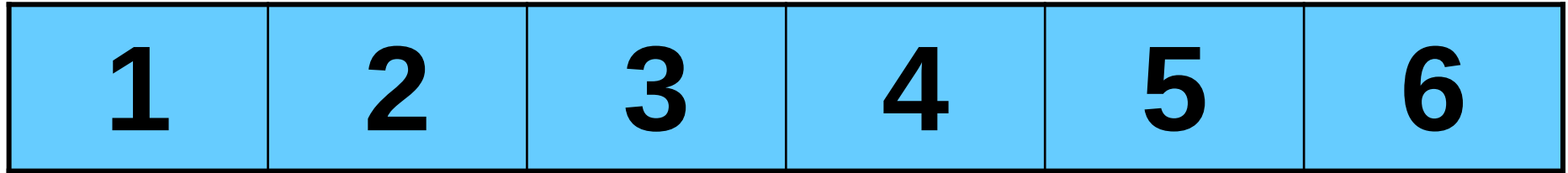


Data Movement



Sorted

Selection Sort



Comparison



Data Movement



Sorted

Selection Sort

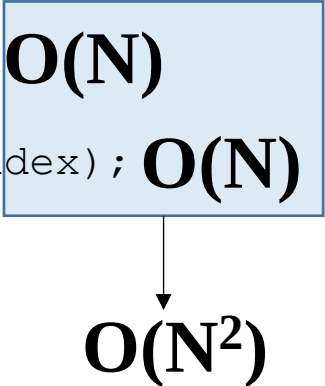
```
template<class ItemType>
int getMinIndex(ItemType values[], int startIndex, int endIndex)
{
    int indexOfMin = startIndex;
    for (int index = startIndex + 1; index <= endIndex; index++)
        if (values[index] < values[indexOfMin])
            indexOfMin = index;
    return indexOfMin;
}

template<class ItemType>
void SelectionSort(ItemType values[], int numValues)
{
    int endIndex = numValues-1;
    int minIndex;
    for (int current = 0; current < endIndex; current++)
    {
        minIndex = getMinIndex(values, current, endIndex);
        Swap(values[current], values[minIndex]);
    }
}
```

Selection Sort

```
template<class ItemType>
int getMinIndex(ItemType values[], int startIndex, int endIndex)
{
    int indexOfMin = startIndex;
    for (int index = startIndex + 1; index <= endIndex; index++)
        if (values[index] < values[indexOfMin])
            indexOfMin = index;
    return indexOfMin;
}

template<class ItemType>
void SelectionSort(ItemType values[], int numValues)
{
    int endIndex = numValues-1;
    int minIndex;
    for (int current = 0; current < endIndex; current++)
    {
        minIndex = getMinIndex(values, current, endIndex);
        Swap(values[current], values[minIndex]);
    }
}
```



The diagram illustrates the time complexity of the Selection Sort algorithm. It features a light blue rectangular box containing the text $O(N)$ and $O(N)$, with an arrow pointing from the second $O(N)$ to $O(N^2)$ below the box. This represents the summation of the complexities of the inner loop and the swap operation across all iterations of the outer loop.

$O(N)$

$O(N)$

$O(N^2)$

Insertion Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Insertion Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Insertion Sort

5	1	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Insertion Sort

1	5	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison



Data Movement



Sorted

Insertion Sort

1	5	3	4	6	2
----------	----------	----------	----------	----------	----------



Comparison

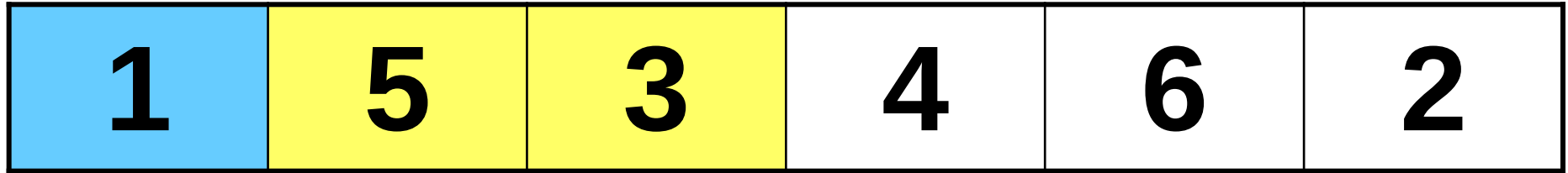


Data Movement



Sorted

Insertion Sort



Comparison

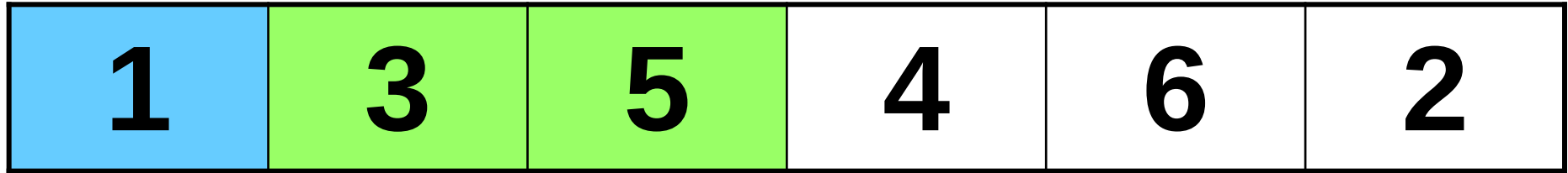


Data Movement



Sorted

Insertion Sort



Comparison

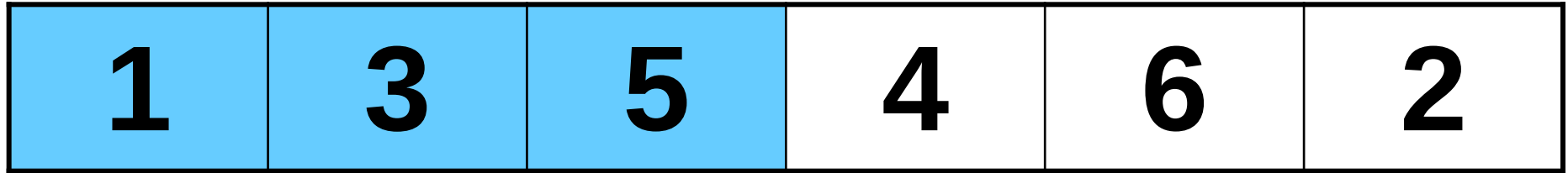


Data Movement



Sorted

Insertion Sort



Comparison

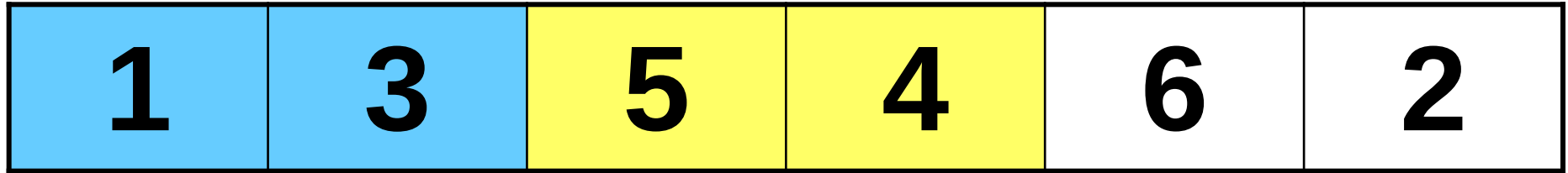


Data Movement



Sorted

Insertion Sort



Comparison

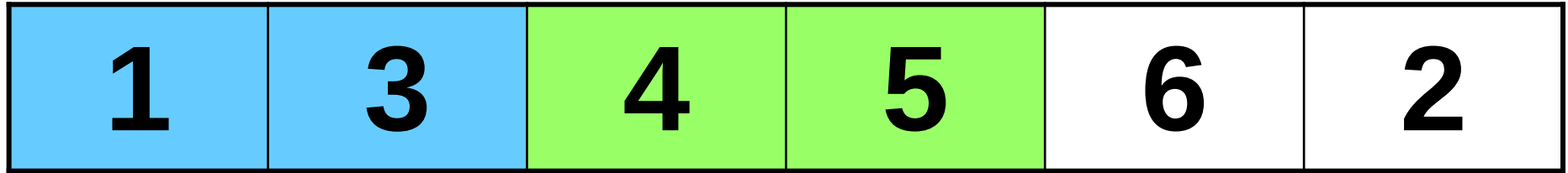


Data Movement



Sorted

Insertion Sort



Comparison



Data Movement



Sorted

Insertion Sort

1	3	4	5	6	2
----------	----------	----------	----------	----------	----------



Comparison

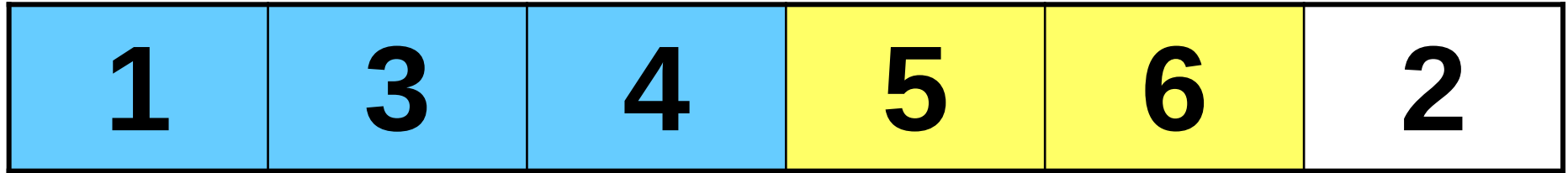


Data Movement



Sorted

Insertion Sort



Comparison

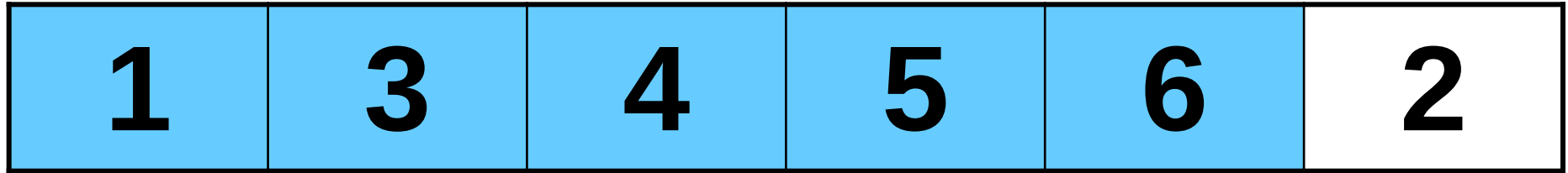


Data Movement



Sorted

Insertion Sort



Comparison

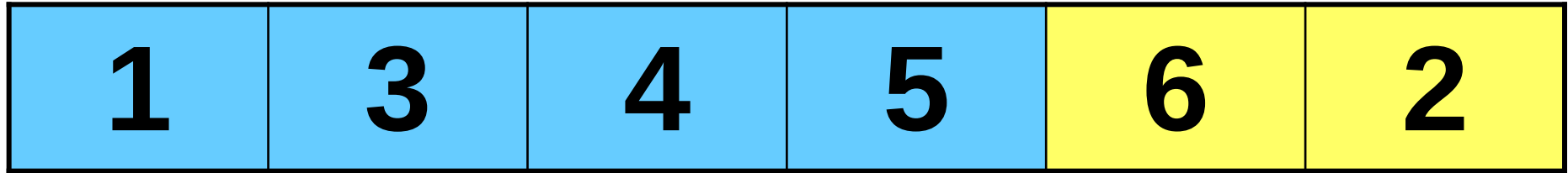


Data Movement



Sorted

Insertion Sort



Comparison

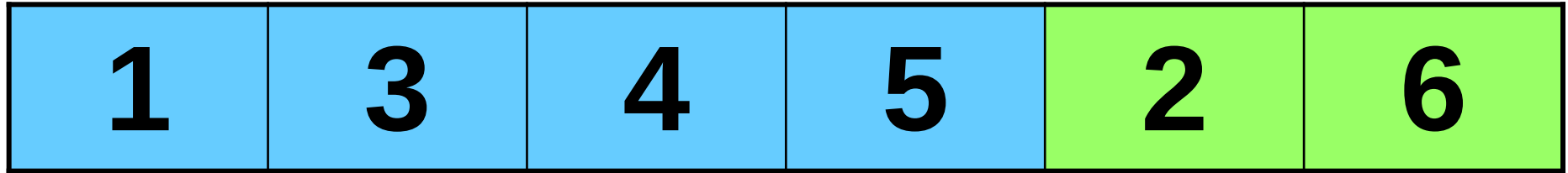


Data Movement



Sorted

Insertion Sort



Comparison

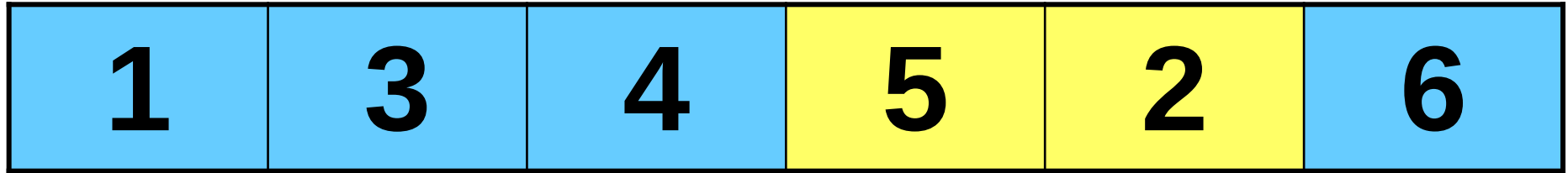


Data Movement



Sorted

Insertion Sort



Comparison

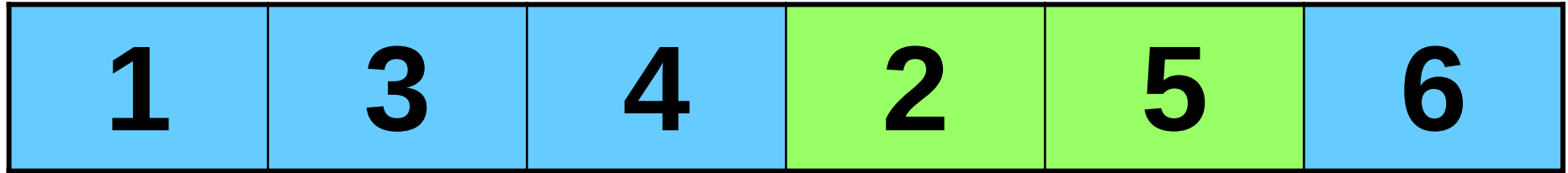


Data Movement



Sorted

Insertion Sort



Comparison

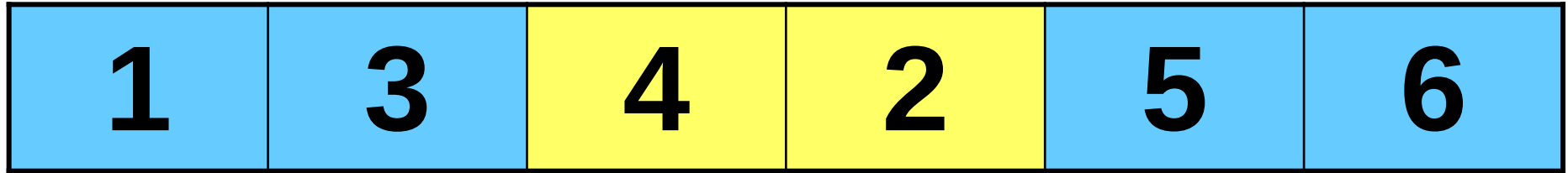


Data Movement



Sorted

Insertion Sort



Comparison

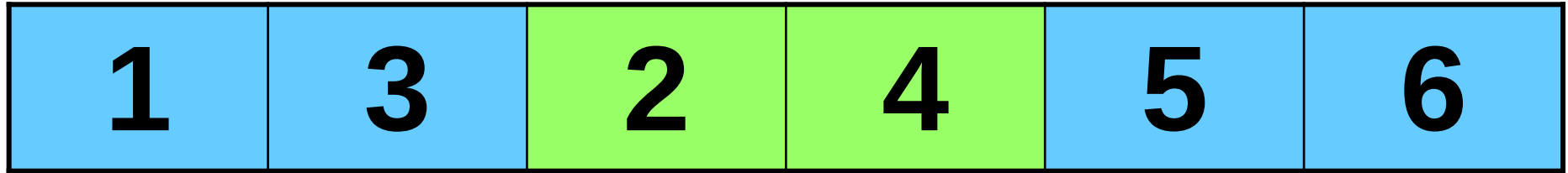


Data Movement



Sorted

Insertion Sort



Comparison

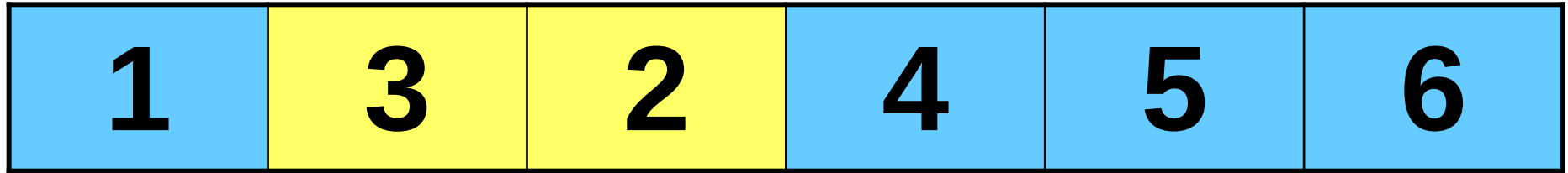


Data Movement



Sorted

Insertion Sort



Comparison

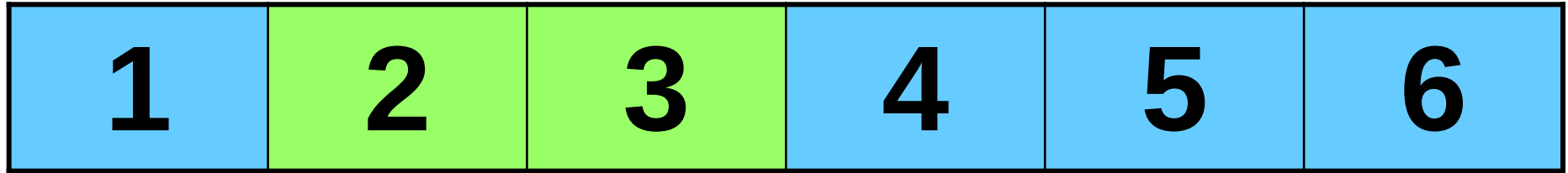


Data Movement



Sorted

Insertion Sort



Comparison

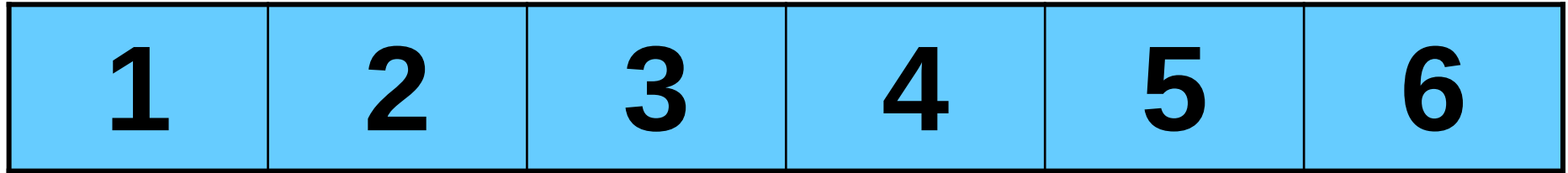


Data Movement



Sorted

Insertion Sort



Comparison



Data Movement



Sorted

Insertion Sort

```
template<class ItemType>
void InsertItem(ItemType values [], int startIndex, int endIndex)
{
    bool finished = false;
    int current = endIndex;
    bool moreToSearch = (current != startIndex);
    while (moreToSearch && !finished)
    {
        if (values[current] < values[current - 1] )
        {
            Swap(values[current], values[current-1]);
            current--;
            moreToSearch = (current != startIndex);
        }
        else finished = true;
    }
}

template<class ItemType>
void InsertionSort(ItemType values[], int numValues)
{
    for (int count = 0; count < numValues; count++)
        InsertItem(values, 0, count);
}
```

Insertion Sort

```
template<class ItemType>
void InsertItem(ItemType values [], int startIndex, int endIndex)
{
    bool finished = false;
    int current = endIndex;
    bool moreToSearch = (current != startIndex);
    while (moreToSearch && !finished)
    {
        if (values[current] < values[current - 1] )
        {
            Swap(values[current], values[current-1]);
            current--;
            moreToSearch = (current != startIndex);
        }
        else finished = true;
    }
}

template<class ItemType>
void InsertionSort(ItemType values[], int numValues)
{
    for (int count = 0; count < numValues; count++)
        InsertItem(values, 0, count);
}
```

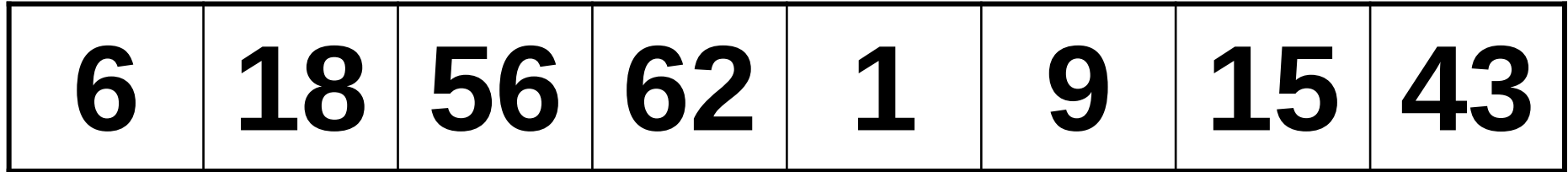
$O(N^2)$

Merge Sort

6	18	56	62	1	9	15	43
----------	-----------	-----------	-----------	----------	----------	-----------	-----------

Merge Sort

Unsorted



6	18	56	62	1	9	15	43
----------	-----------	-----------	-----------	----------	----------	-----------	-----------

Sorted

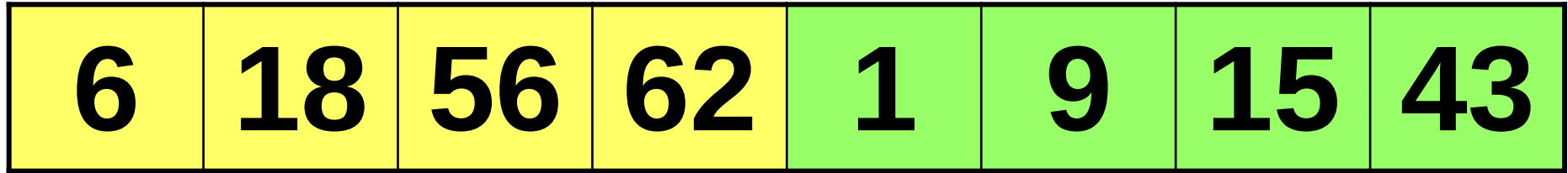
Sorted

Merge Sort

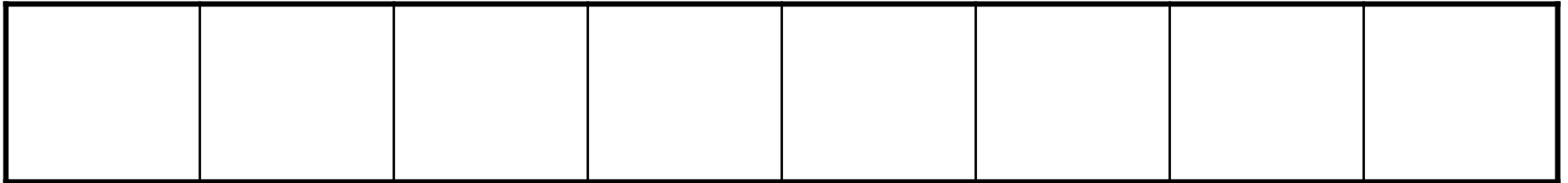
6	18	56	62	1	9	15	43
----------	-----------	-----------	-----------	----------	----------	-----------	-----------

Merging

Merge Sort



Merging



Left half

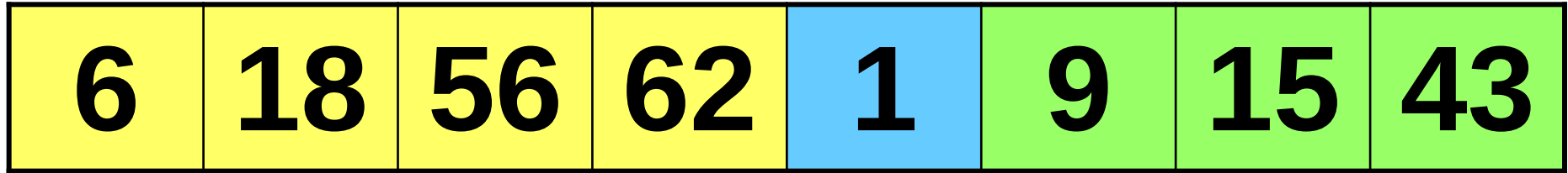


Right half

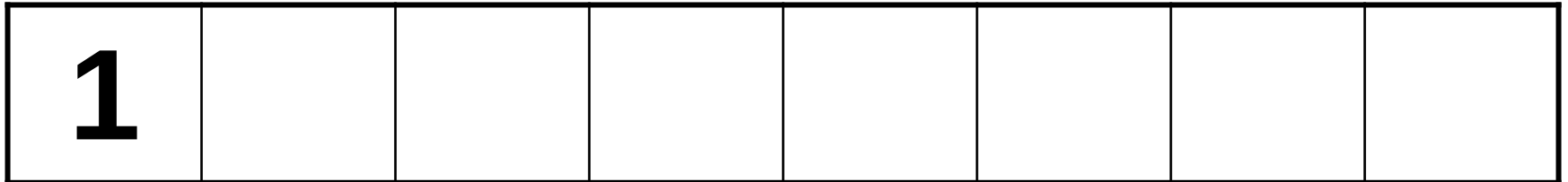


Minimum between first elements in both halves

Merge Sort



Merging



Left half

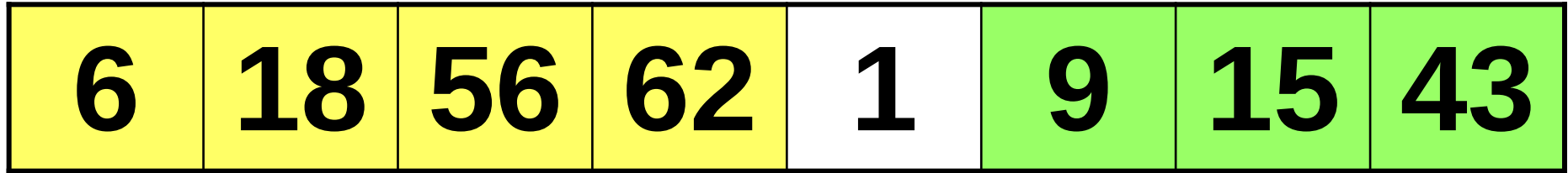


Right half

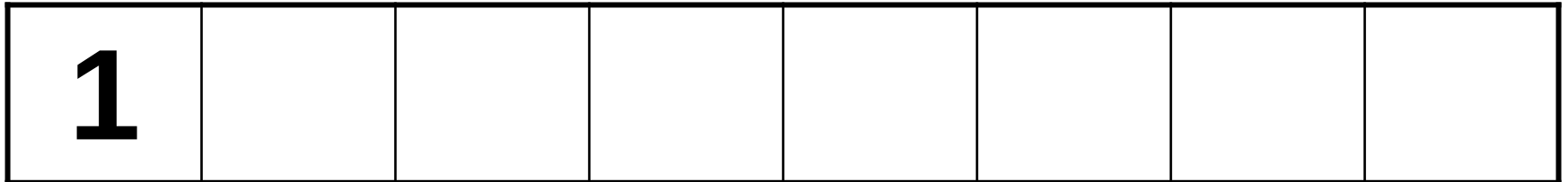


Minimum between first elements in both halves

Merge Sort



Merging



Left half

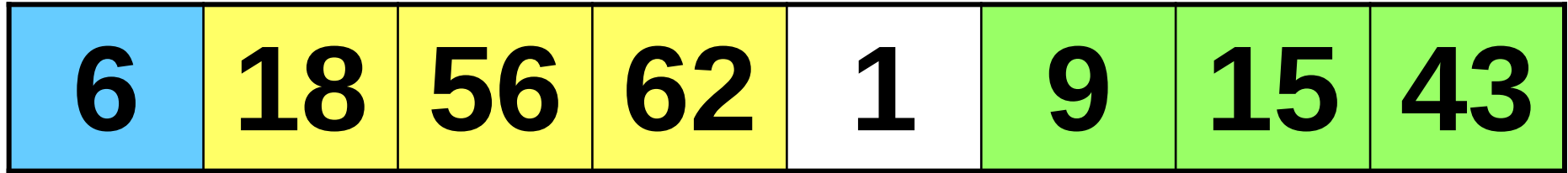


Right half

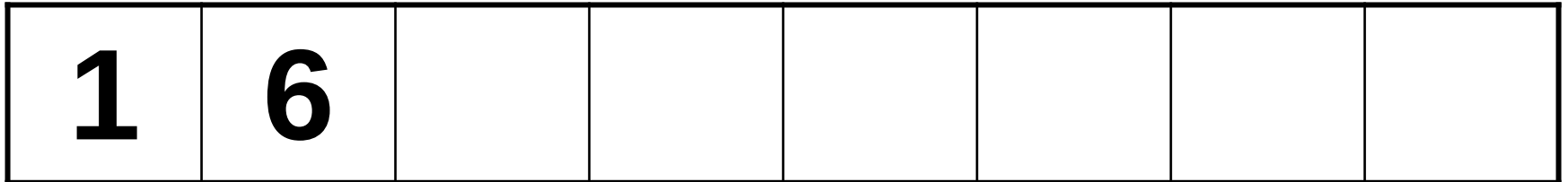


Minimum between first elements in both halves

Merge Sort



Merging



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
---	----	----	----	---	---	----	----

Merging

1	6						
---	---	--	--	--	--	--	--



Left half

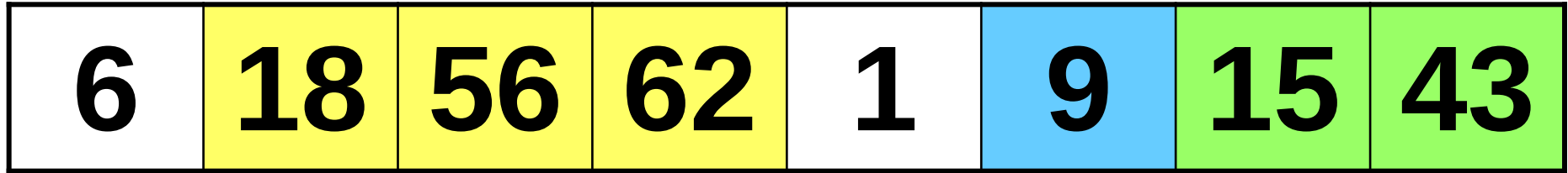


Right half

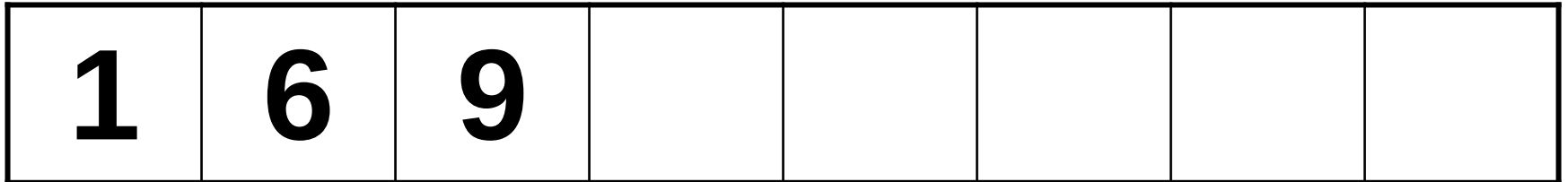


Minimum between first elements in both halves

Merge Sort



Merging



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
---	----	----	----	---	---	----	----

Merging

1	6	9					
---	---	---	--	--	--	--	--



Left half

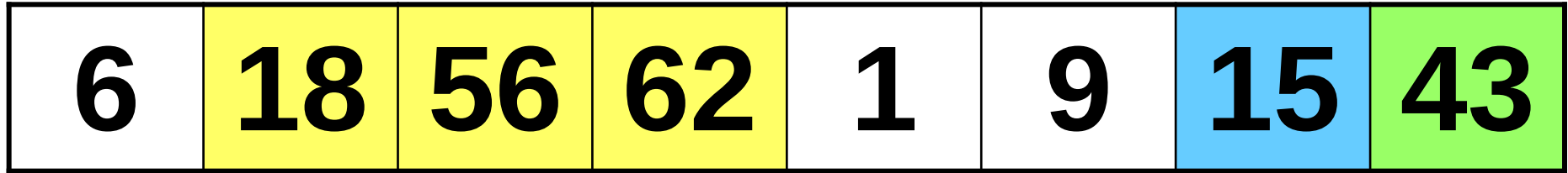


Right half

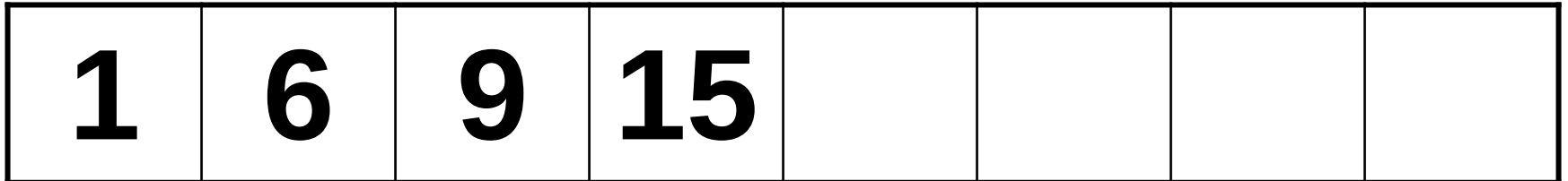


Minimum between first elements in both halves

Merge Sort



Merging



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
---	----	----	----	---	---	----	----

Merging

1	6	9	15				
---	---	---	----	--	--	--	--



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
---	----	----	----	---	---	----	----

Merging

1	6	9	15	18			
---	---	---	----	----	--	--	--



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
---	----	----	----	---	---	----	----

Merging

1	6	9	15	18			
---	---	---	----	----	--	--	--



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
---	----	----	----	---	---	----	----

Merging

1	6	9	15	18	43		
---	---	---	----	----	----	--	--



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
---	----	----	----	---	---	----	----

Merging

1	6	9	15	18	43		
---	---	---	----	----	----	--	--



Left half



Right half



Minimum between first elements in both halves

Merge Sort

6	18	56	62	1	9	15	43
----------	-----------	-----------	-----------	----------	----------	-----------	-----------

Merging

1	6	9	15	18	43	56	62
----------	----------	----------	-----------	-----------	-----------	-----------	-----------



Left half



Right half



Minimum between first elements in both halves

Merge Sort

1	6	9	15	18	43	56	62
----------	----------	----------	-----------	-----------	-----------	-----------	-----------

Merging

↑	↑	↑	↑	↑	↑	↑	↑
1	6	9	15	18	43	56	62



Left half



Right half



Minimum between first elements in both halves

Merge Sort

1

6

9

15

18

43

56

62

Merge Sort

1	6	9	15	18	43	56	62
----------	----------	----------	-----------	-----------	-----------	-----------	-----------

$O(N)$

Merge Sort

- **Merging**

```
template<class ItemType>
void Merge(ItemType values[], int leftFirst, int leftLast,
          int rightFirst, int rightLast)
{
    ItemType tempArray[MAX_ITEMS];
    int index = leftFirst, saveFirst = leftFirst;

    while ((leftFirst <= leftLast) && (rightFirst <= rightLast))
    {
        if (values[leftFirst] < values[rightFirst])
        {
            tempArray[index] = values[leftFirst];
            leftFirst++;
        }
        else
        {
            tempArray[index] = values[rightFirst];
            rightFirst++;
        }
        index++;
    }
}
```


Merge Sort

- **Merging**

```
while (leftFirst <= leftLast)
{
    tempArray[index] = values[leftFirst];
    leftFirst++;
    index++;
}

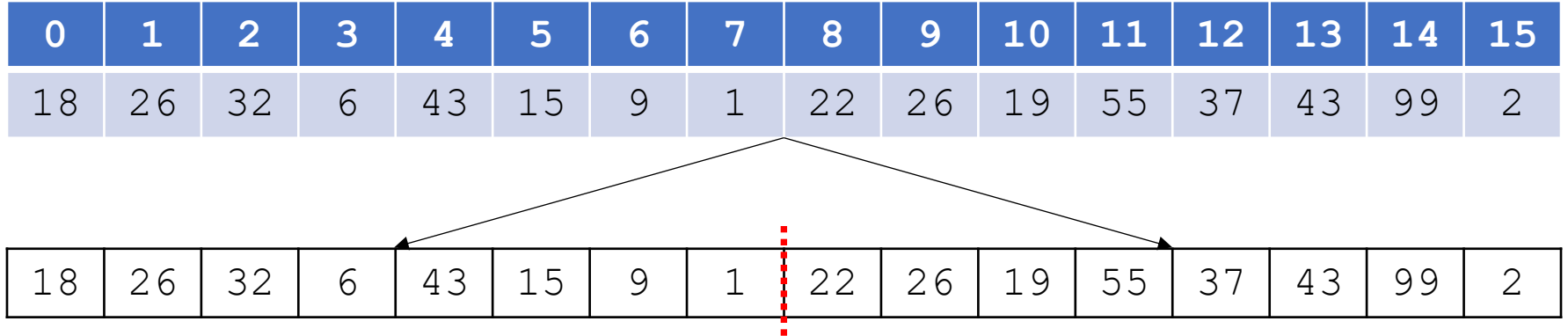
while (rightFirst <= rightLast)
{
    tempArray[index] = values[rightFirst];
    rightFirst++;
    index++;
}

for (index = saveFirst; index <= rightLast; index++)
    values[index] = tempArray[index];
}
```

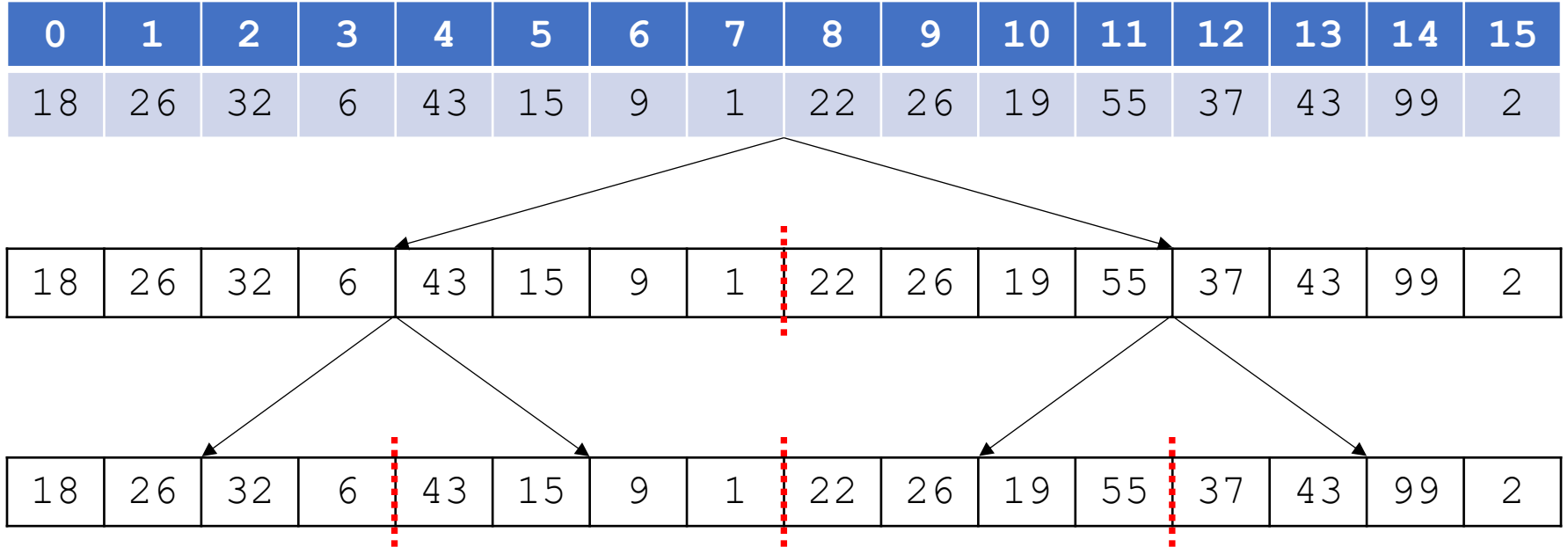
Merge Sort

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
18	26	32	6	43	15	9	1	22	26	19	55	37	43	99	2

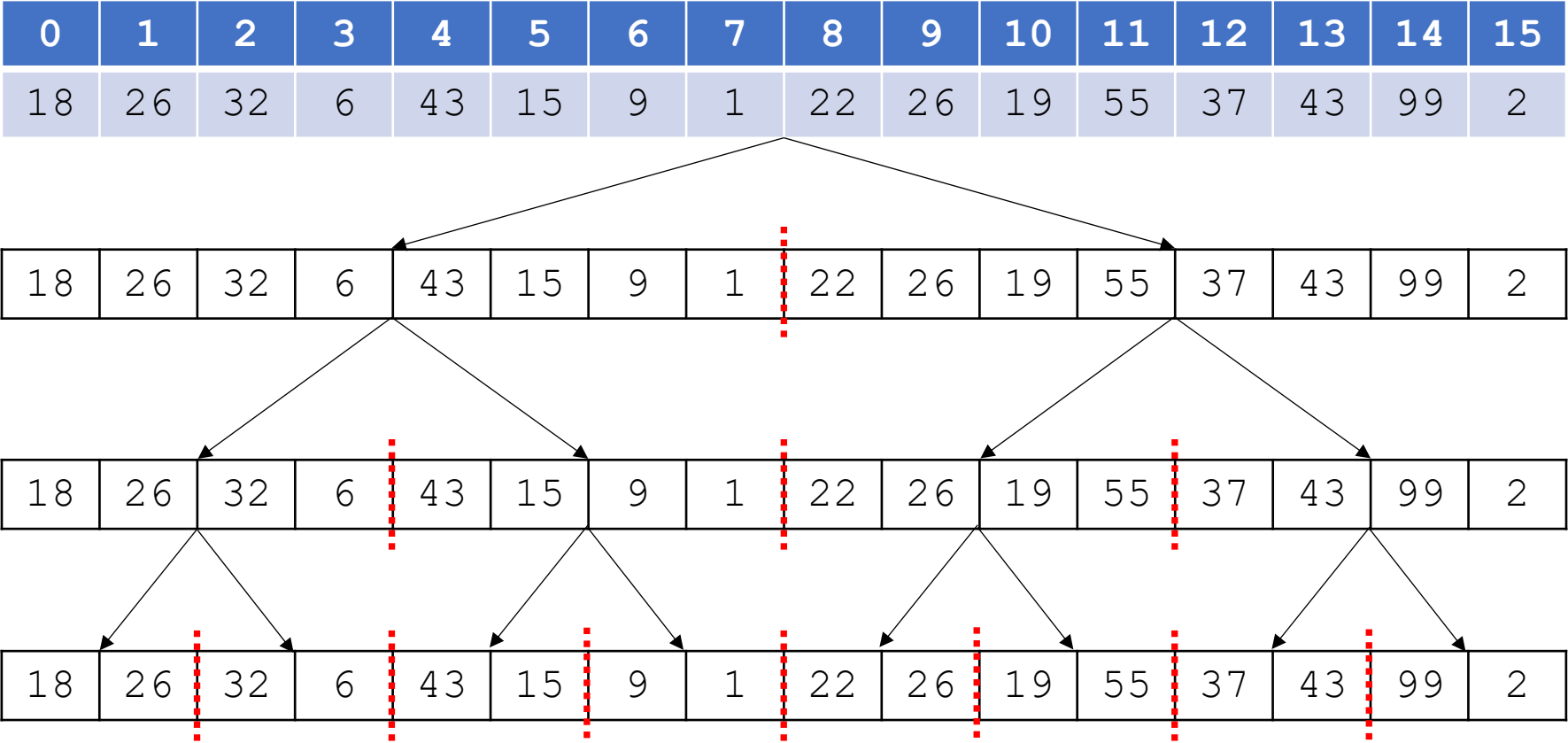
Merge Sort



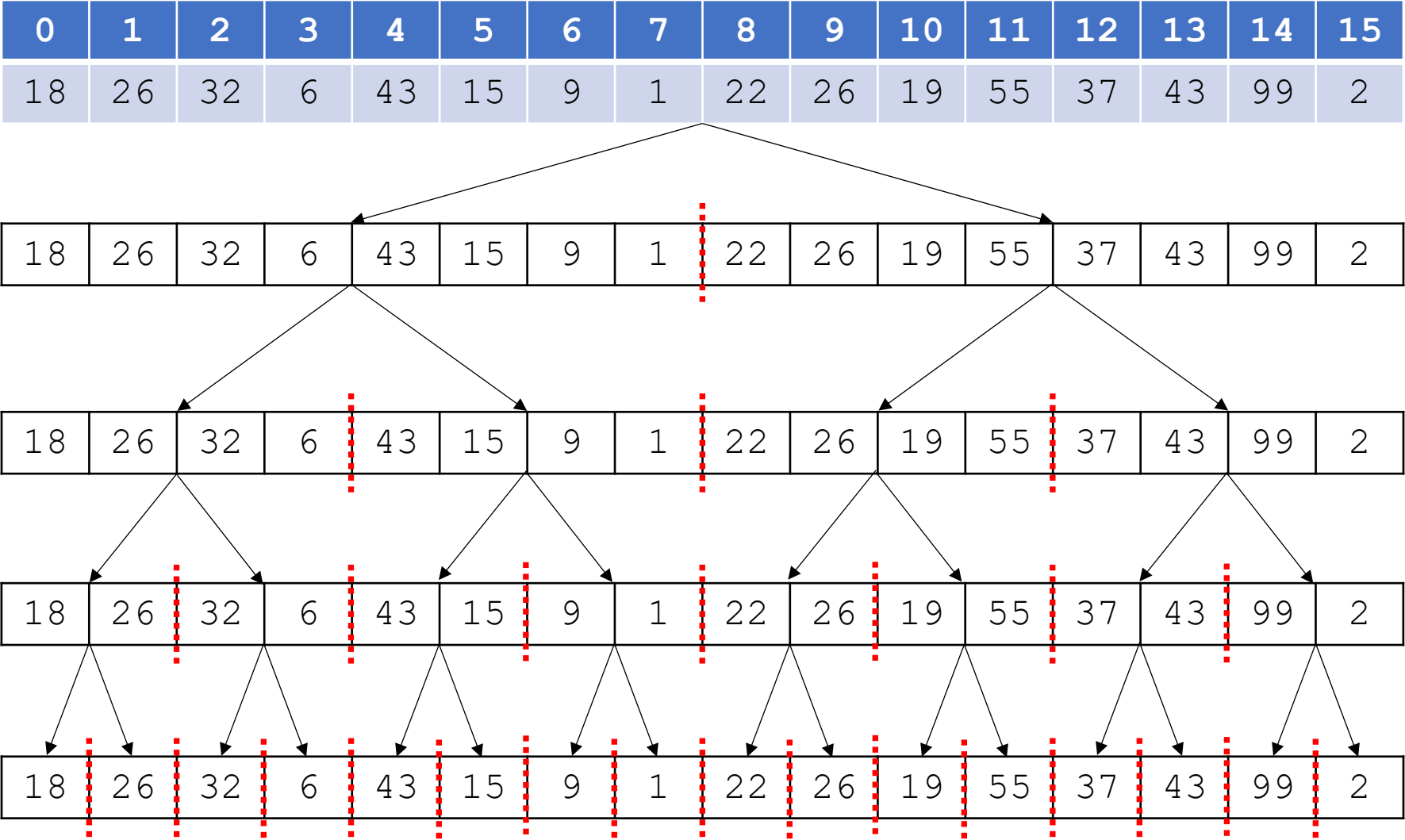
Merge Sort



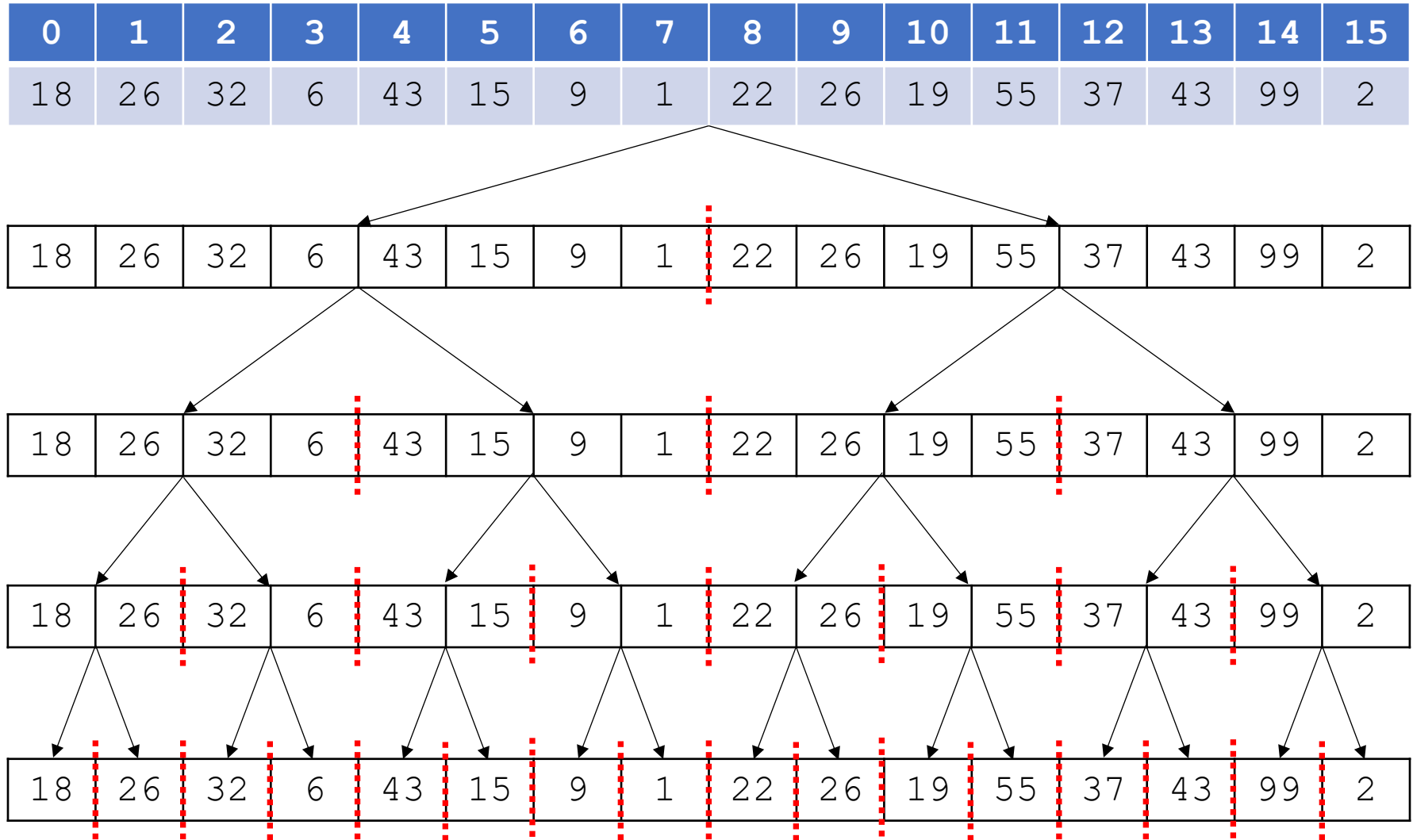
Merge Sort



Merge Sort

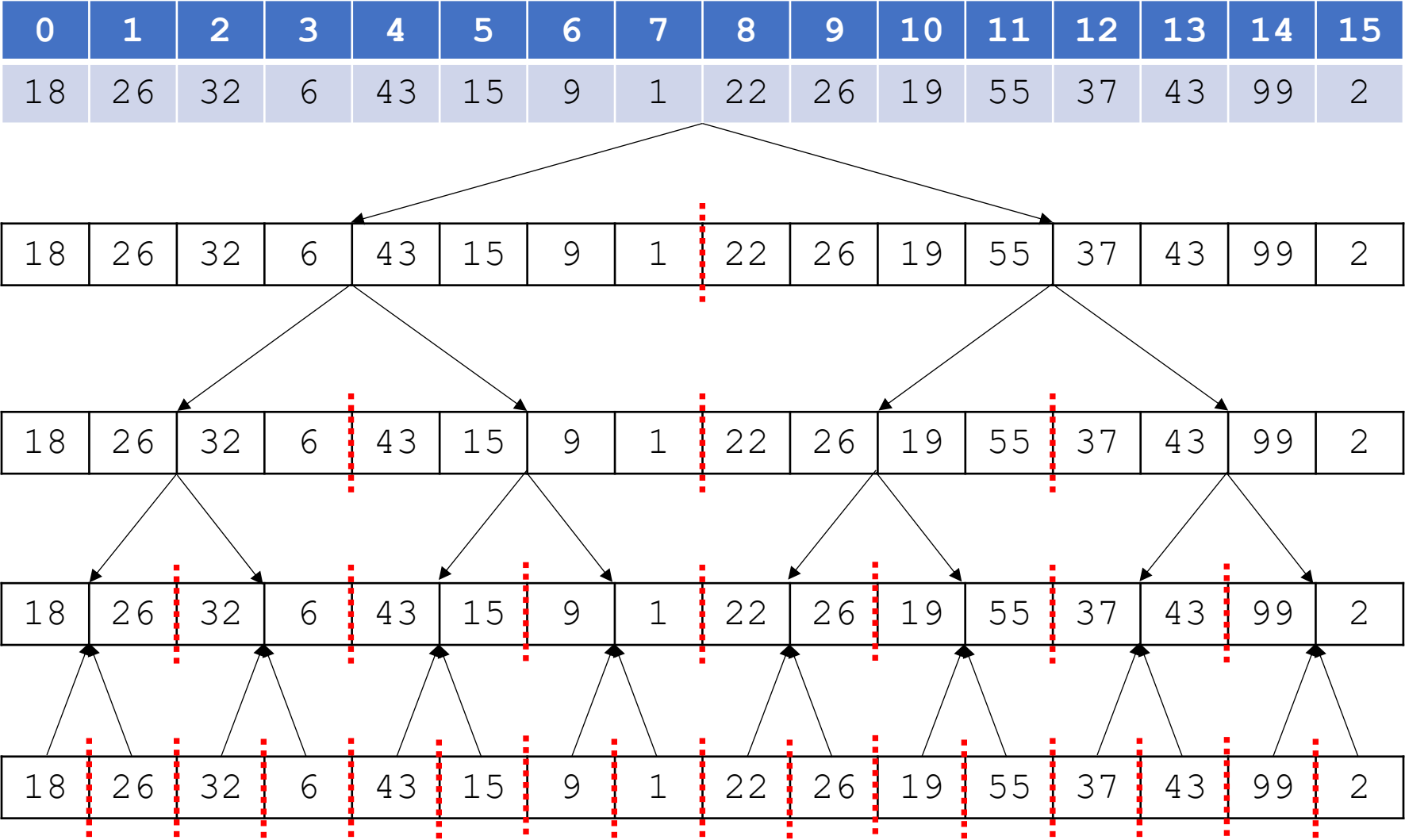


Merge Sort

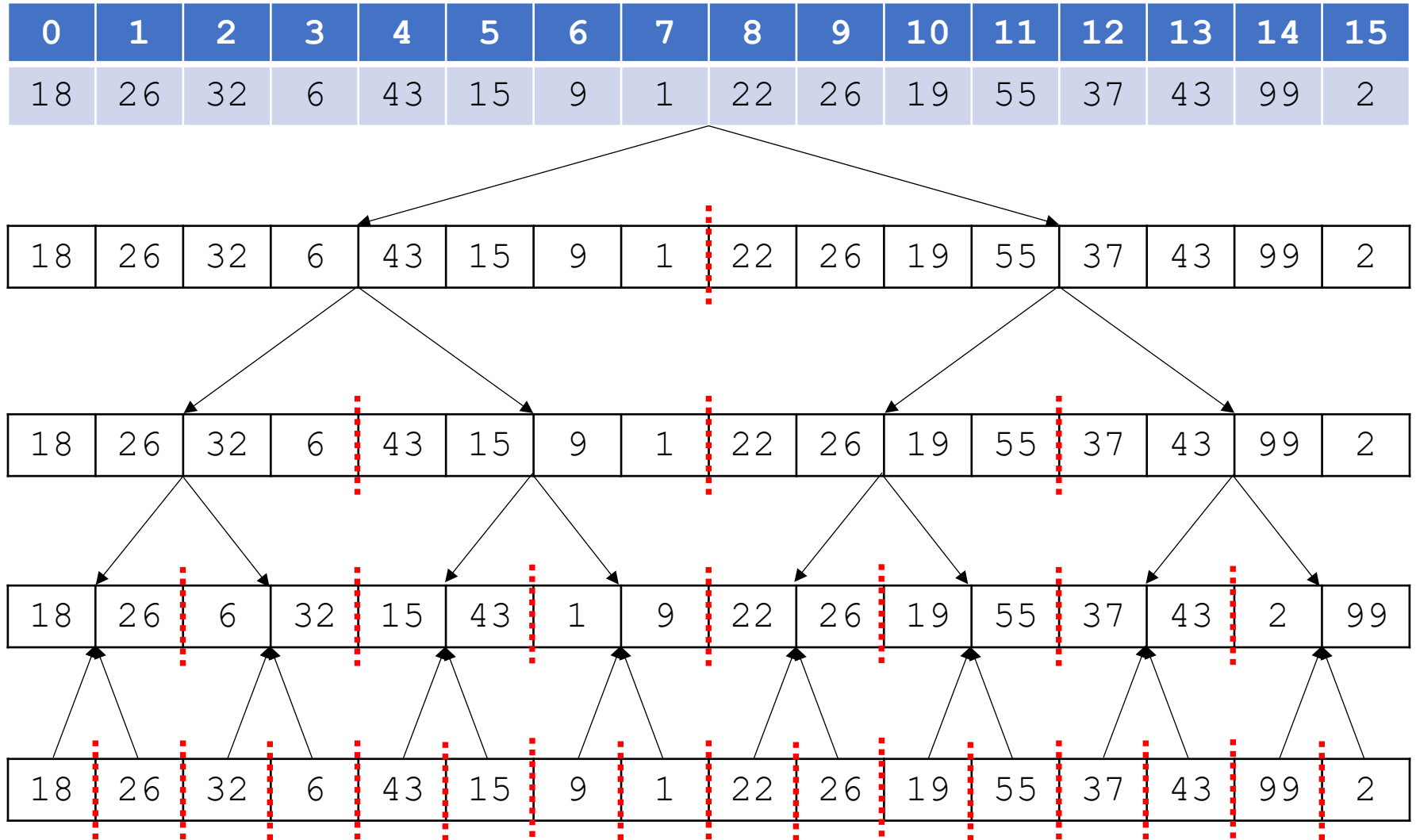


$O(\log N)$ for dividing

Merge Sort

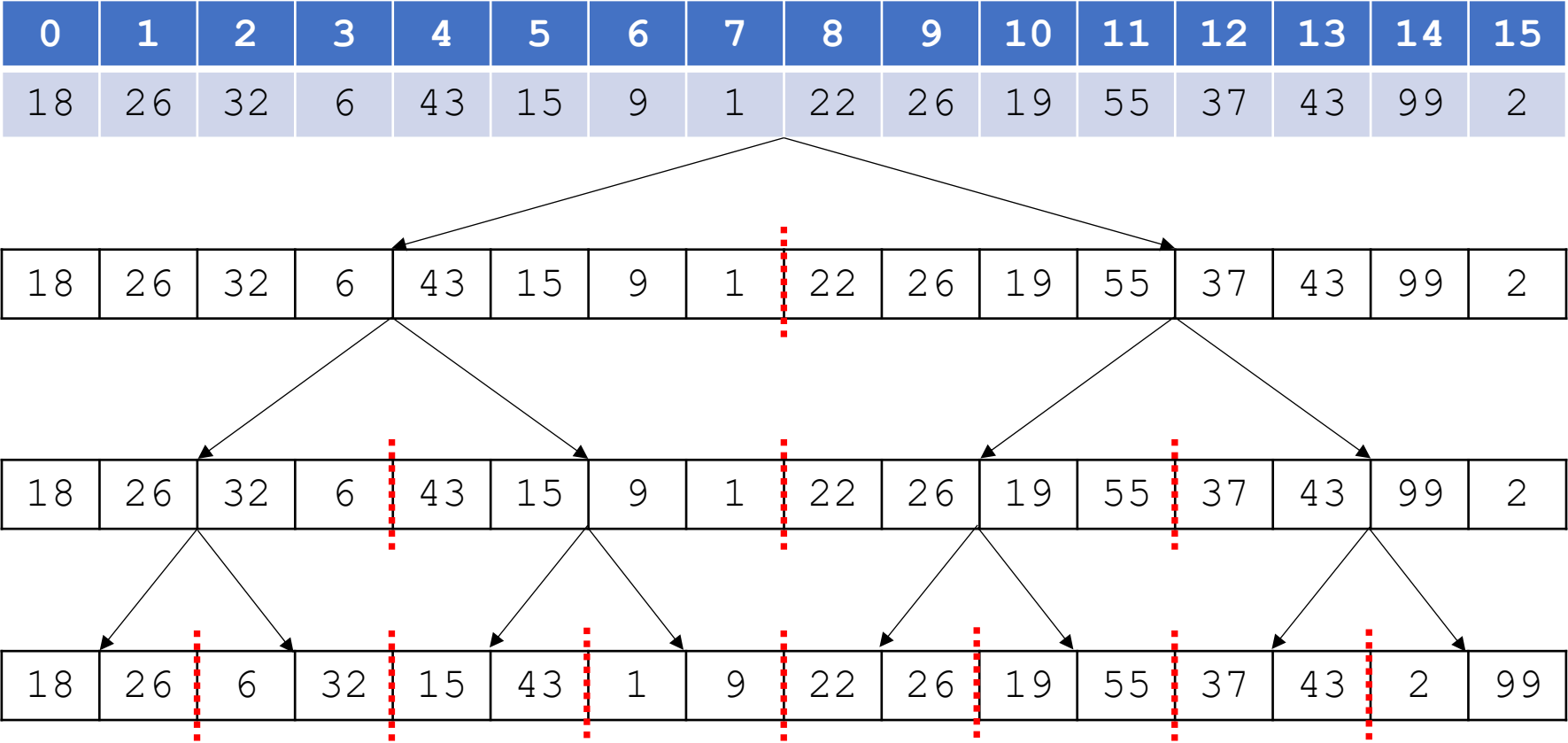


Merge Sort

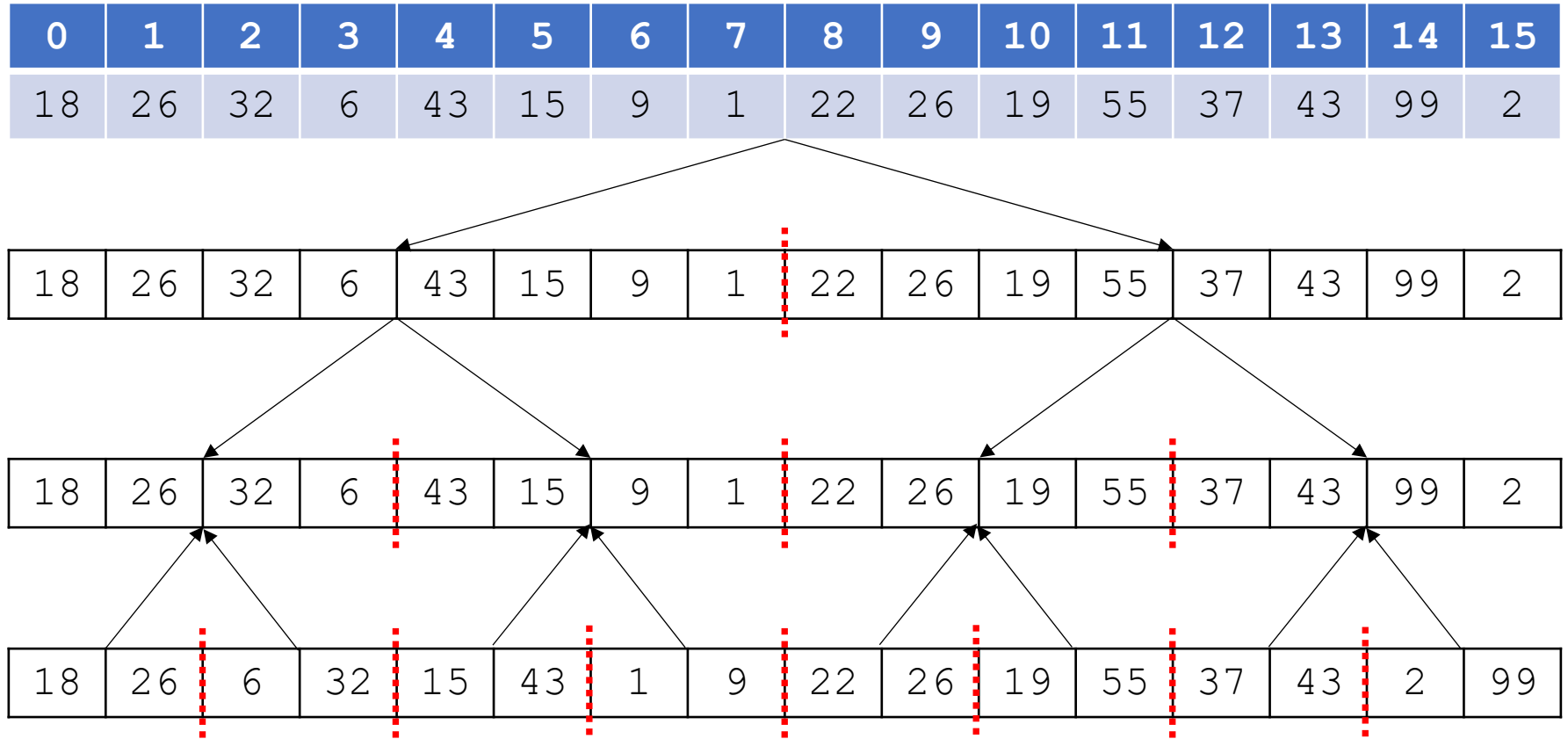


$O(N)$ for merging

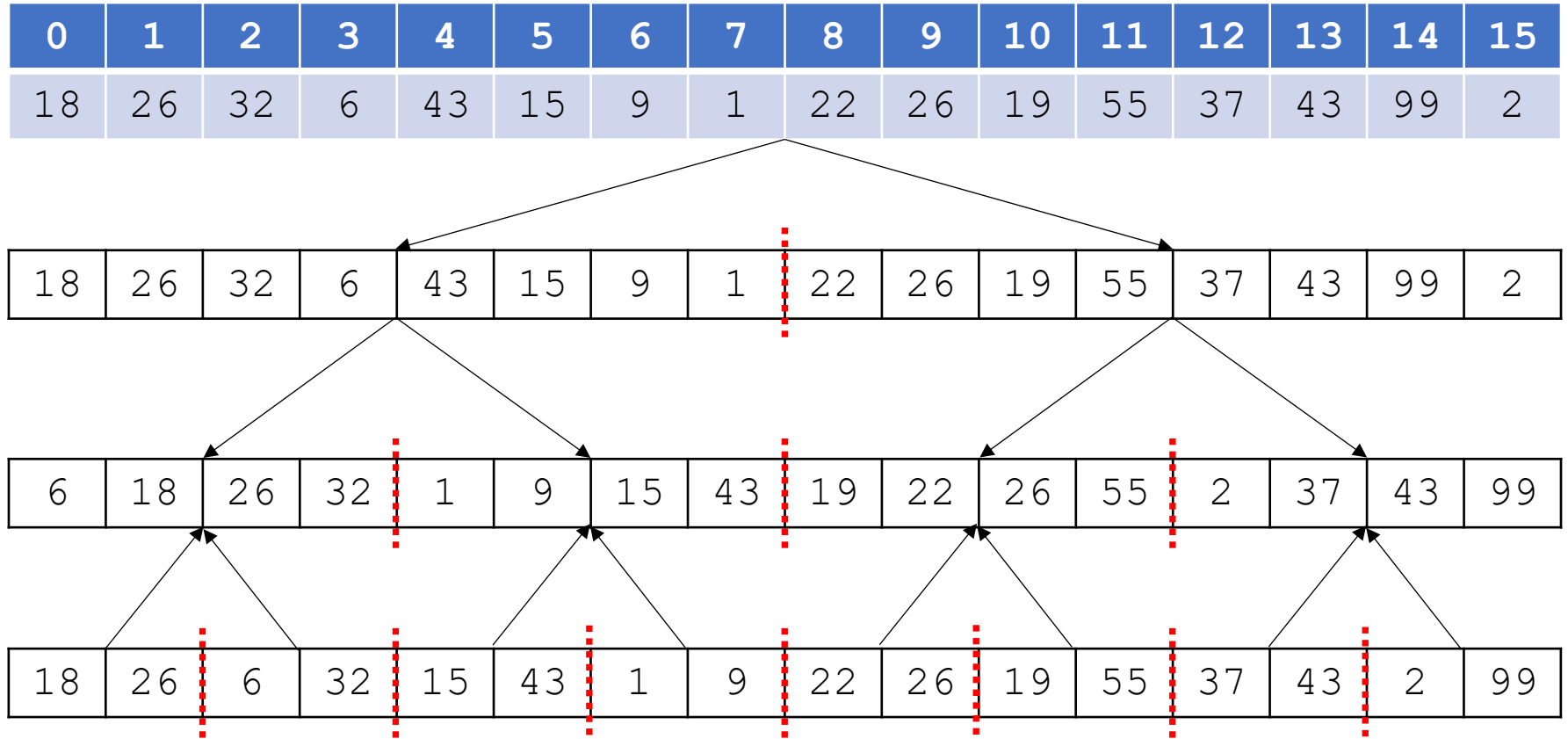
Merge Sort



Merge Sort

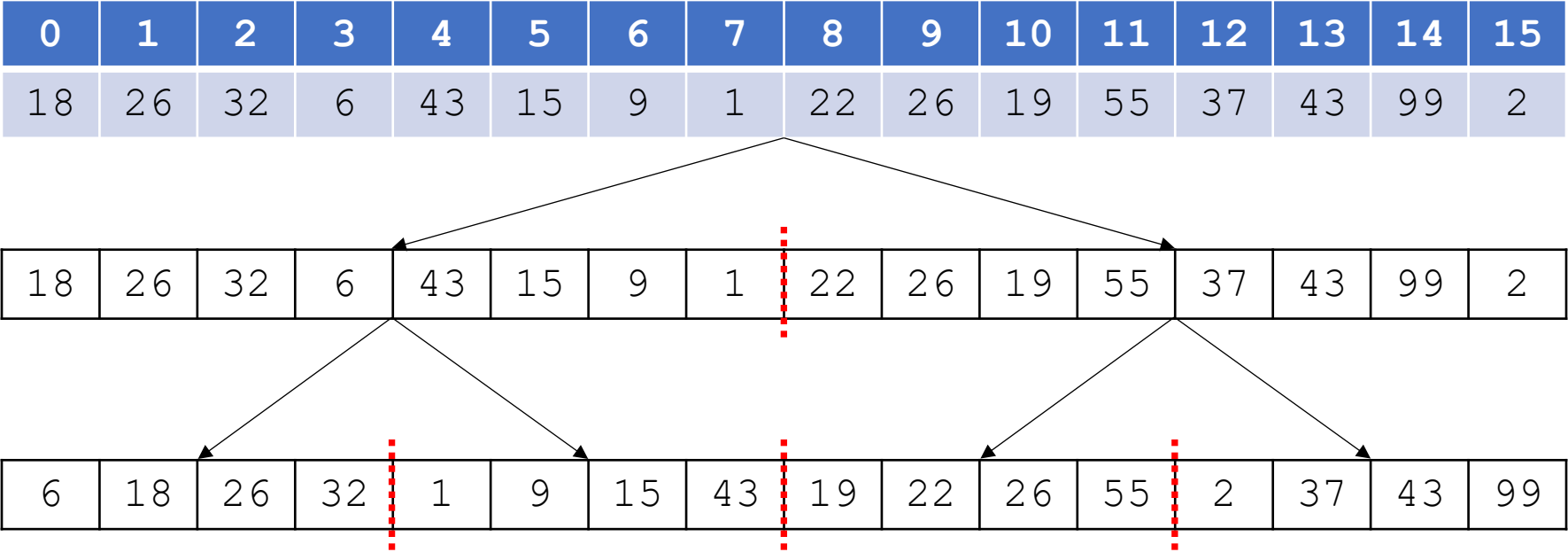


Merge Sort

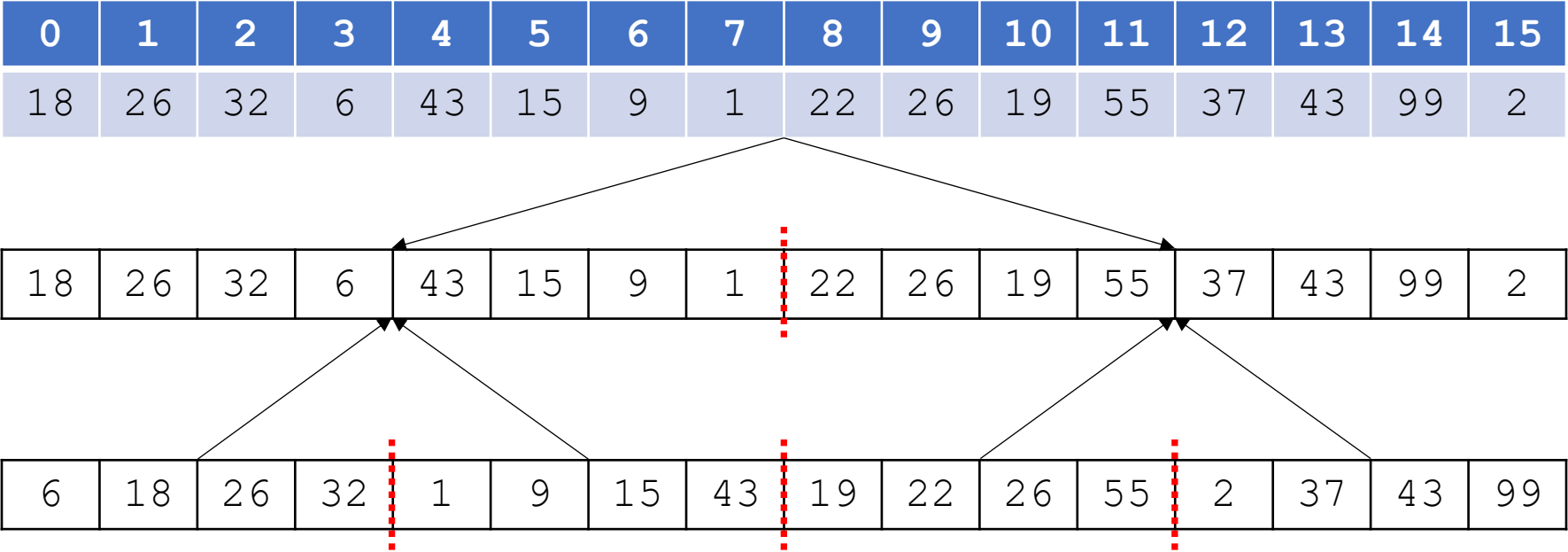


$O(N)$ for merging

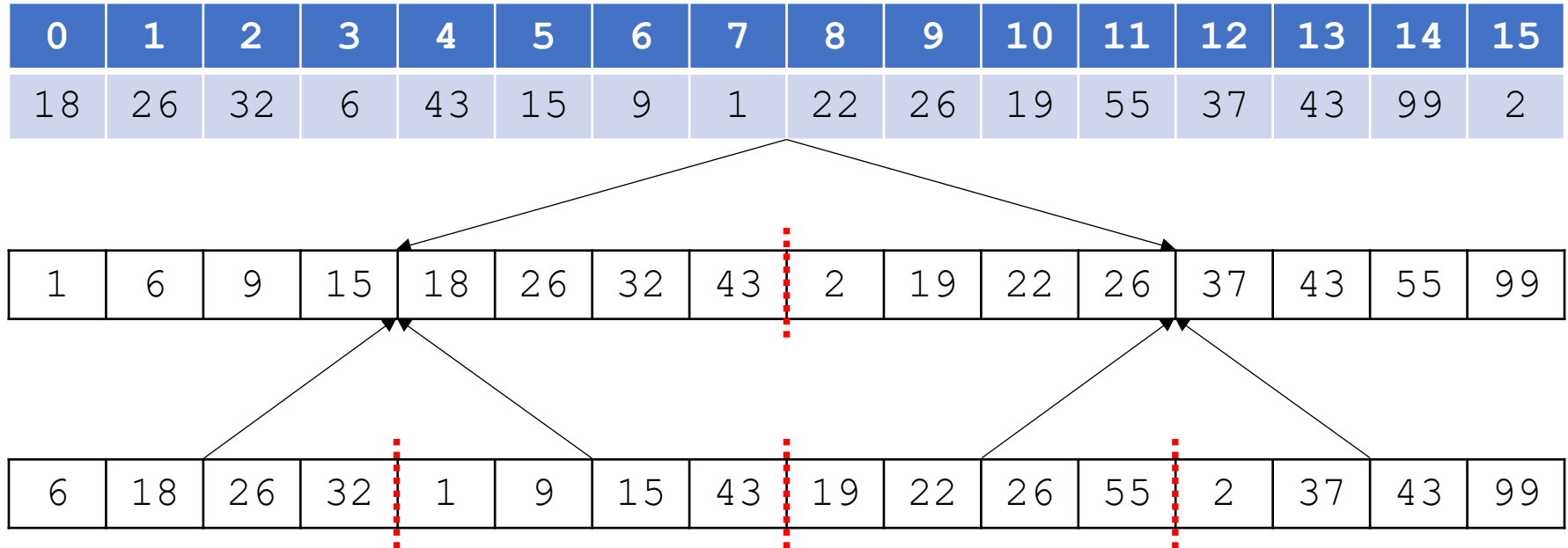
Merge Sort



Merge Sort

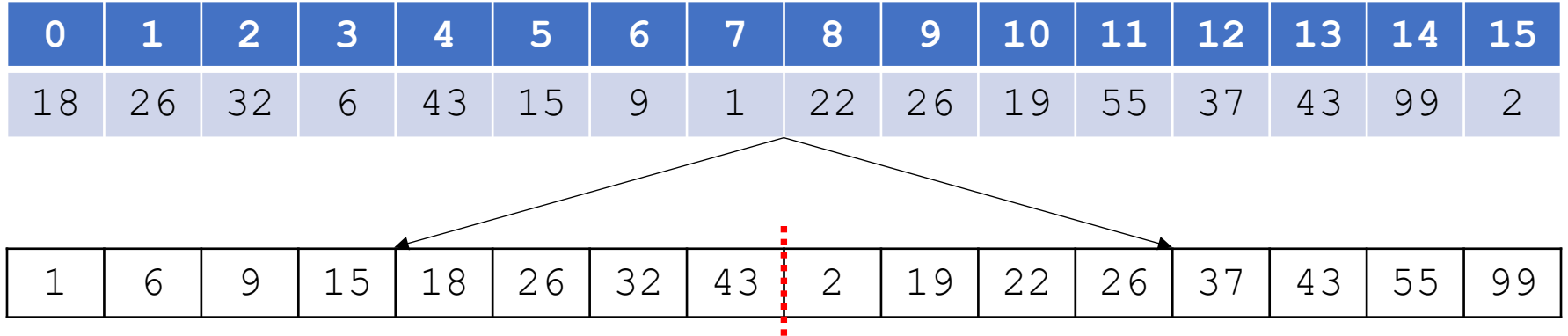


Merge Sort

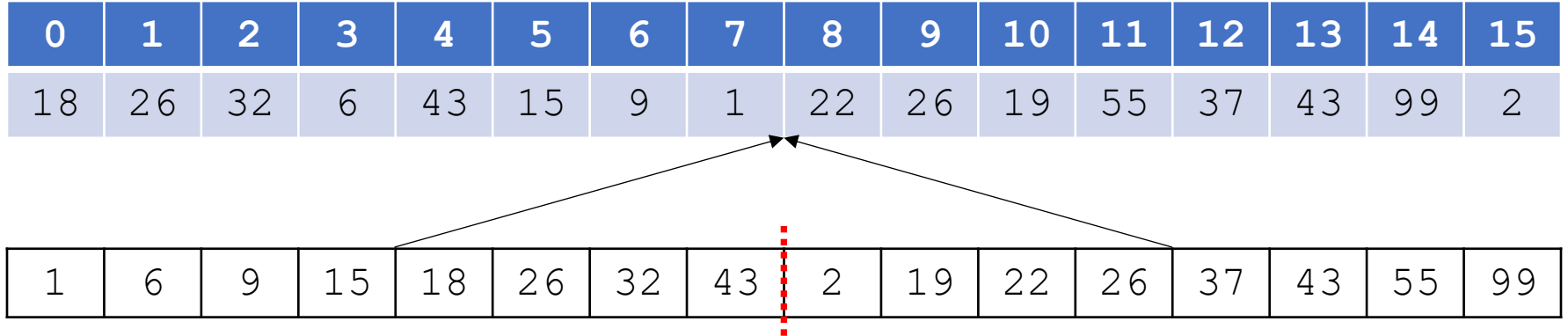


$O(N)$ for merging

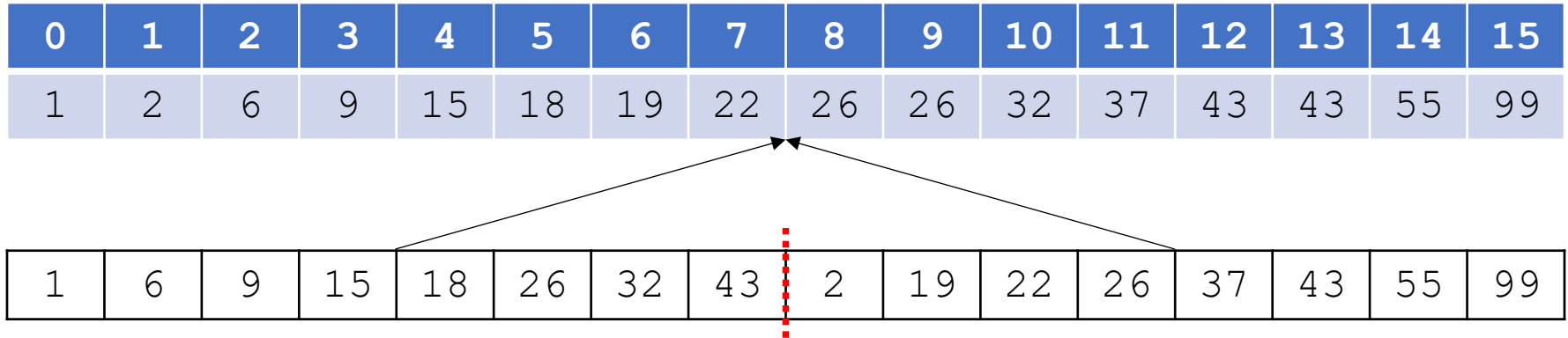
Merge Sort



Merge Sort



Merge Sort



$O(N)$ for merging

Merge Sort

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	2	6	9	15	18	19	22	26	26	32	37	43	43	55	99

Merge Sort

```
template<class ItemType>
void MergeSort(ItemType values[], int first, int last)
{
    if (first < last)
    {
        int middle = (first + last) / 2;
        MergeSort(values, first, middle);
        MergeSort(values, middle + 1, last);
        Merge(values, first, middle, middle + 1, last);
    }
}
```

Merge Sort

```
template<class ItemType>
void MergeSort(ItemType values[], int first, int last)
{
    if (first < last)
    {
        int middle = (first + last) / 2;
        MergeSort(values, first, middle);
        MergeSort(values, middle + 1, last);
        Merge(values, first, middle, middle + 1, last);
    }
}
```

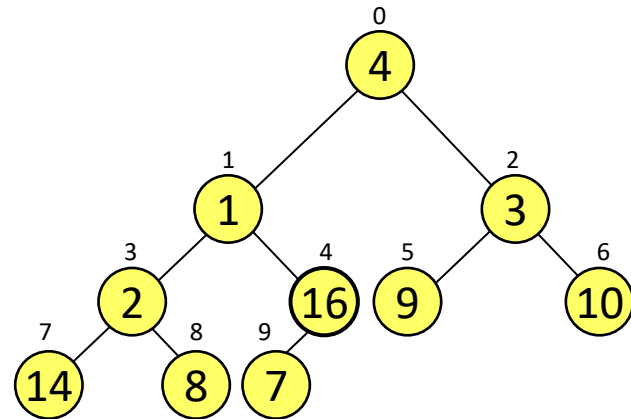
O(NlogN)

Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

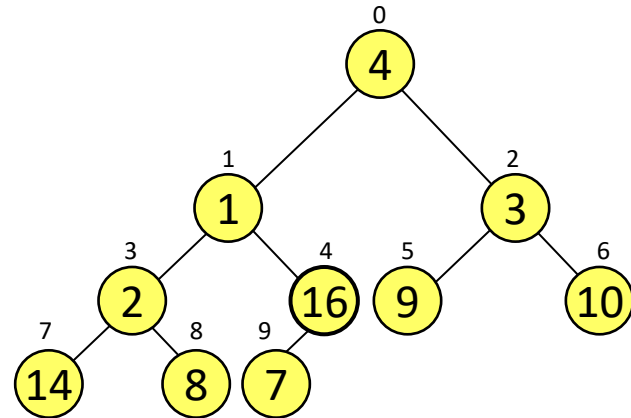


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown($A, i, n-1$)

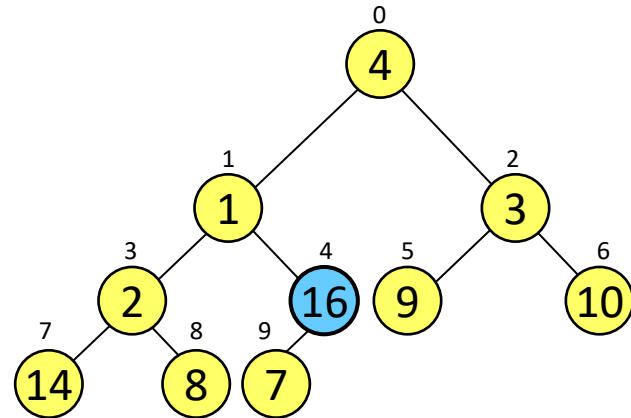


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown($A, i, n-1$)

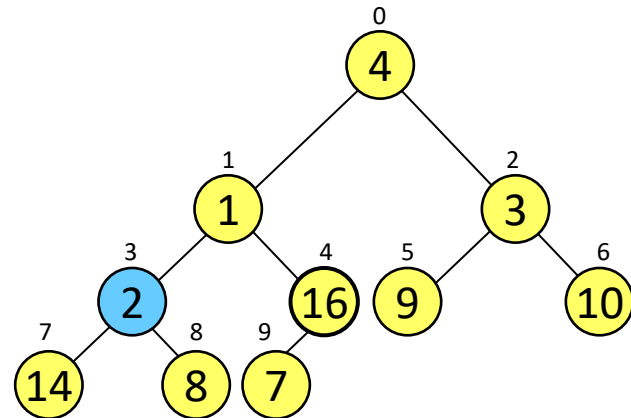


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown($A, i, n-1$)

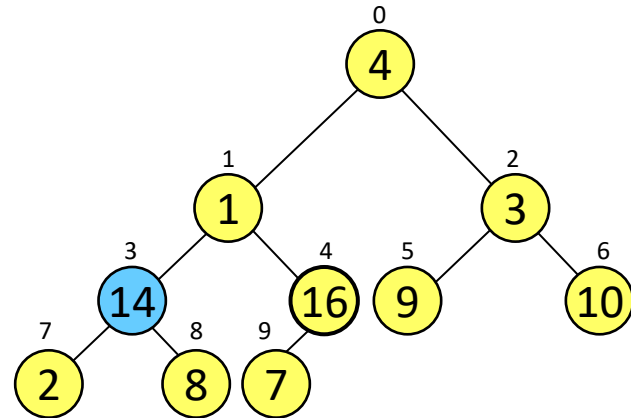


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	14	16	9	10	2	8	7

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown($A, i, n-1$)

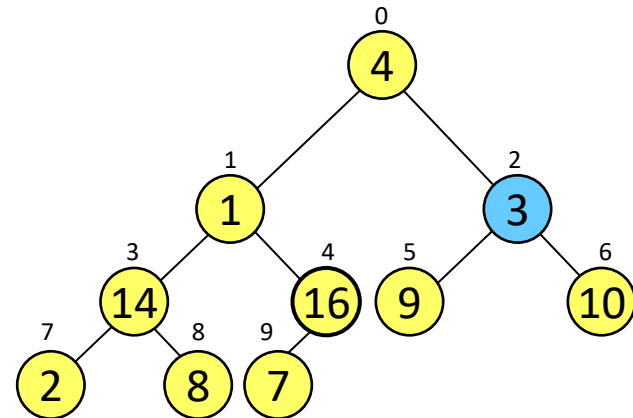


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	14	16	9	10	2	8	7

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)

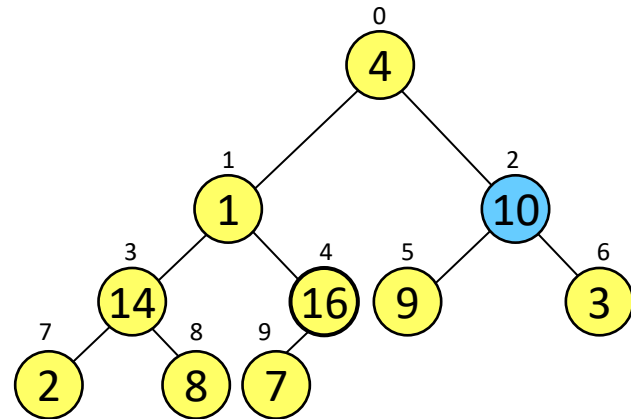


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	10	14	16	9	3	2	8	7

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)

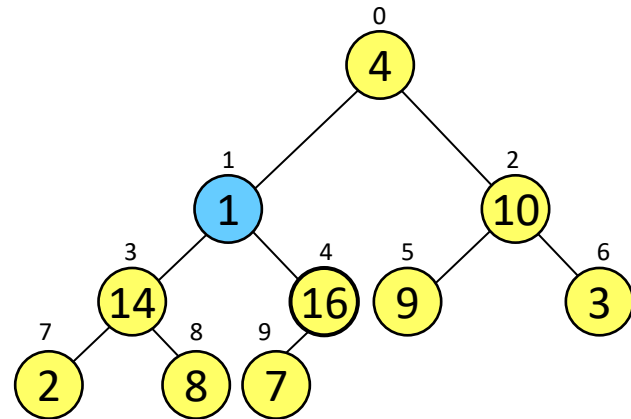


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	10	14	16	9	3	2	8	7

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown($A, i, n-1$)

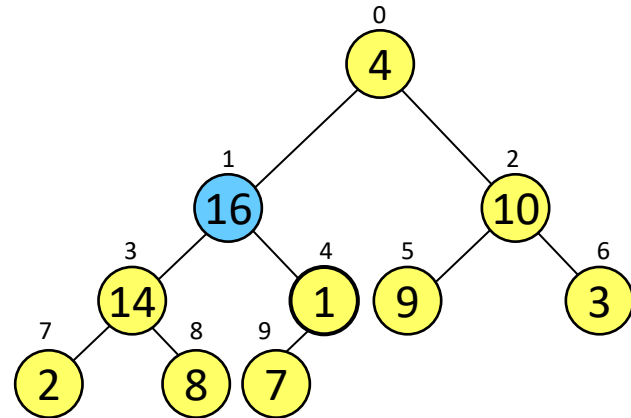


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	16	10	14	1	9	3	2	8	7

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)

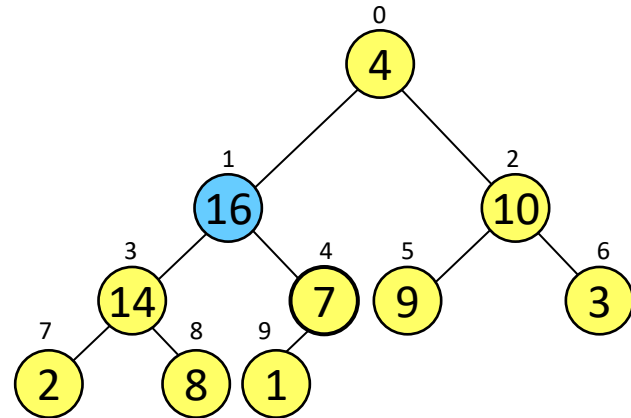


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	16	10	14	7	9	3	2	8	1

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)

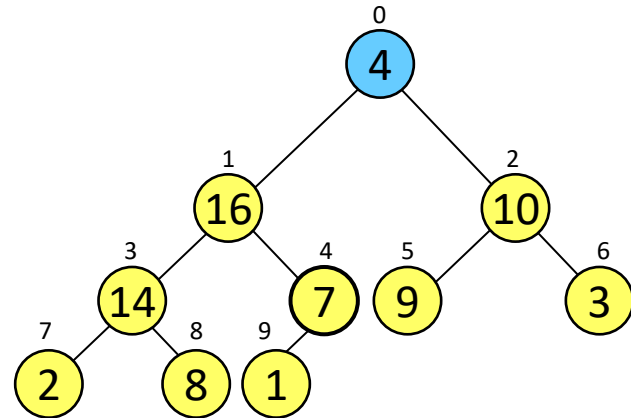


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	16	10	14	7	9	3	2	8	1

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)

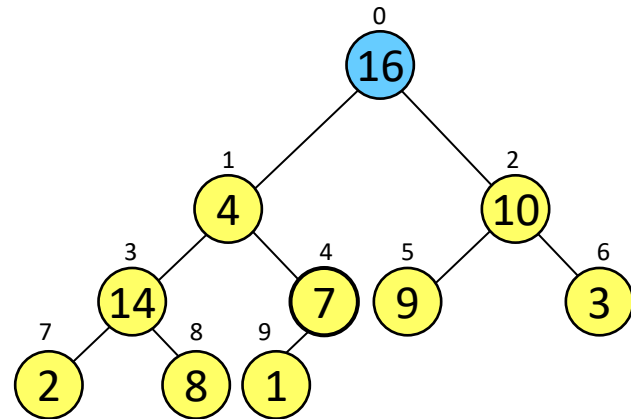


Heap Sort

0	1	2	3	4	5	6	7	8	9
16	4	10	14	7	9	3	2	8	1

Alg: BUILD-MAX-HEAP(A)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown($A, i, n-1$)

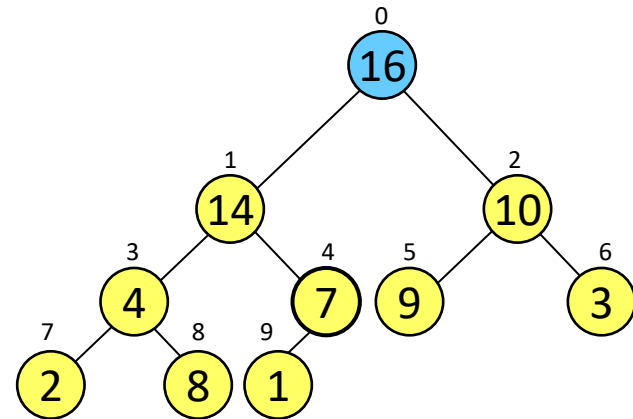


Heap Sort

0	1	2	3	4	5	6	7	8	9
16	14	10	4	7	9	3	2	8	1

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)

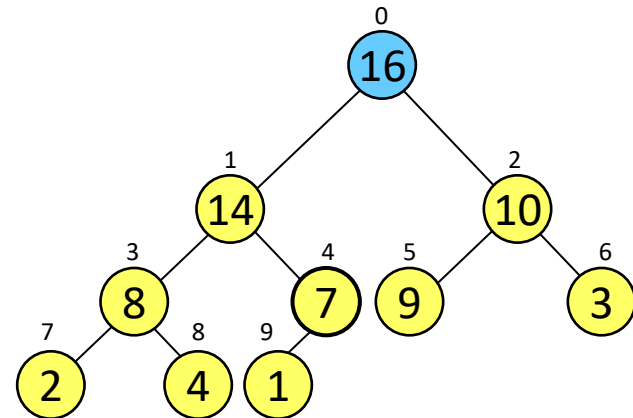


Heap Sort

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)

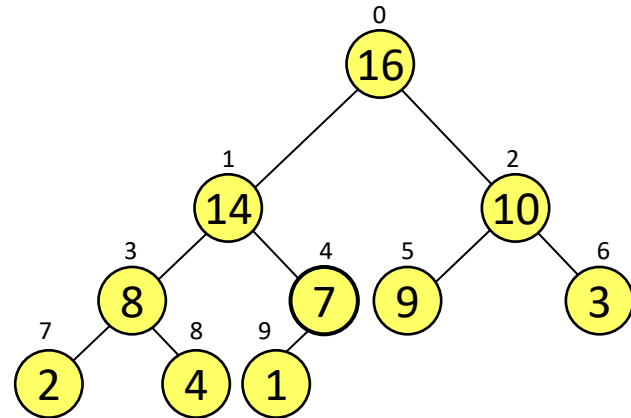


Heap Sort

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Alg: BUILD-MAX-HEAP(*A*)

1. $n = \text{length}[A] - 1$
2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0
3. **do** ReheapDown(*A*, *i*, *n*-1)



Heap Sort

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Alg: BUILD-MAX-HEAP(*A*)

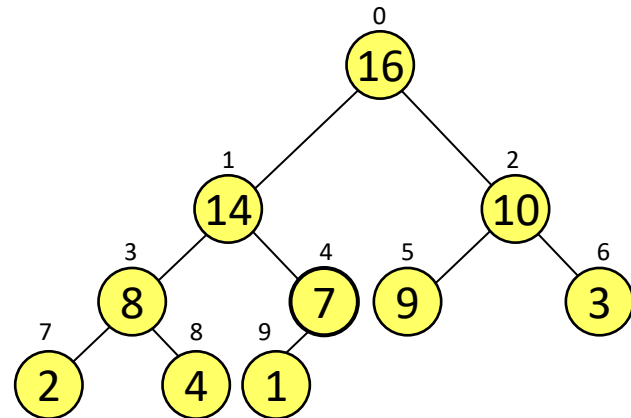
$O(N)$

1. $n = \text{length}[A] - 1$

2. **for** $i \leftarrow \lfloor (n-1)/2 \rfloor$ **downto** 0

3. **do** ReheapDown(*A*, *i*, *n*-1)

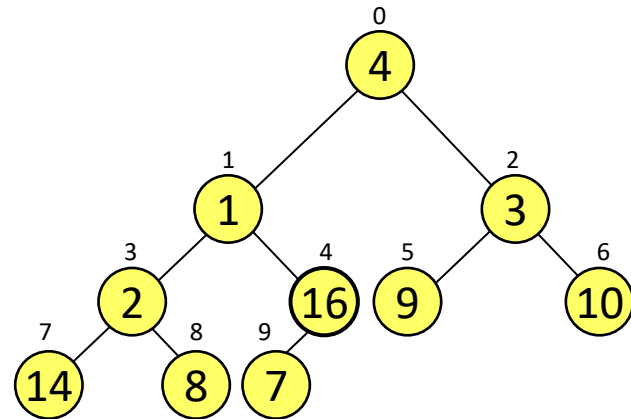
$O(\log N)$



Time complexity: $O(N \log N)$

Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

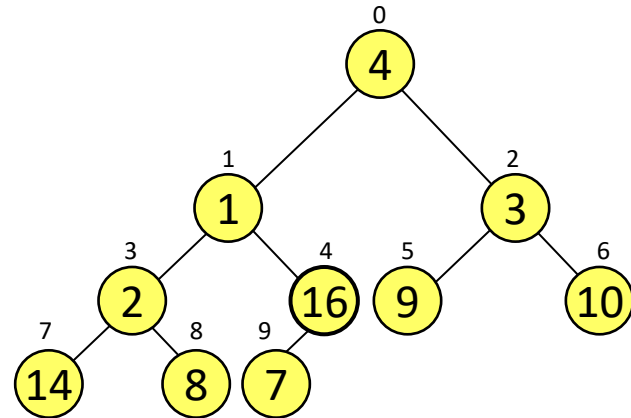


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

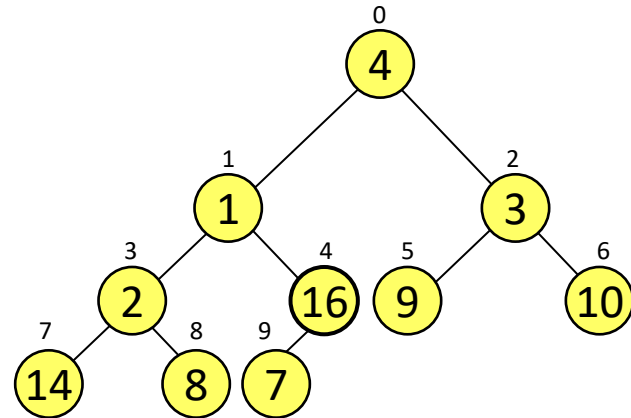


Heap Sort

0	1	2	3	4	5	6	7	8	9
4	1	3	2	16	9	10	14	8	7

Alg: HEAPSORT(A)

1. **BUILD-MAX-HEAP(A)**
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown($A, 0, i-1$)

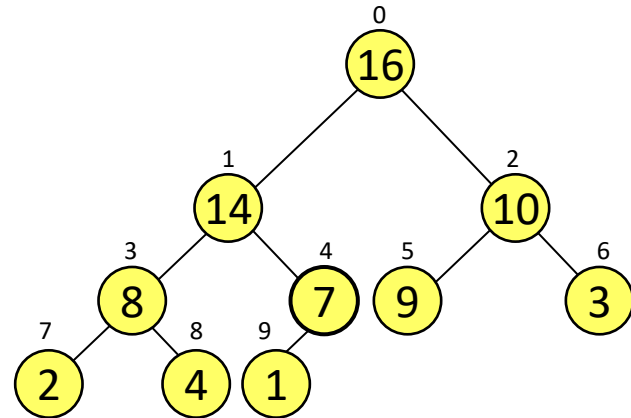


Heap Sort

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Alg: HEAPSORT(A)

1. **BUILD-MAX-HEAP(A)**
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown($A, 0, i-1$)

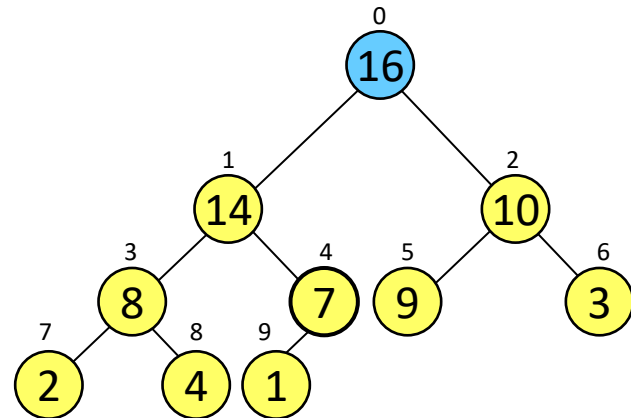


Heap Sort

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

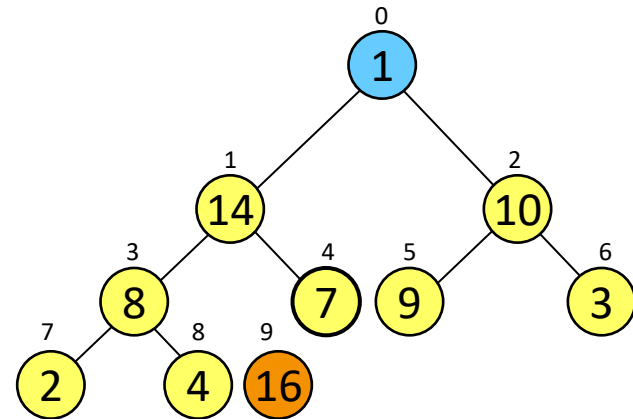


Heap Sort

0	1	2	3	4	5	6	7	8	9
1	14	10	8	7	9	3	2	4	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

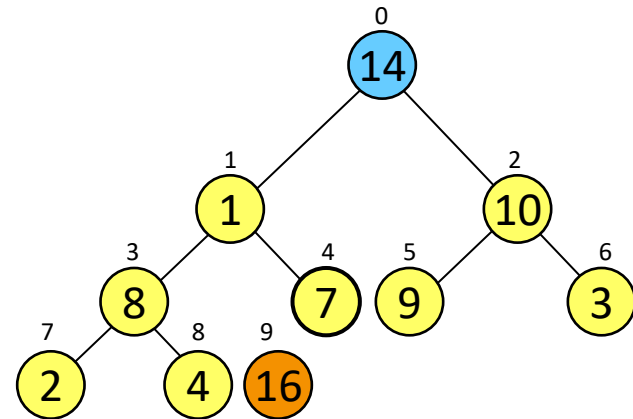


Heap Sort

0	1	2	3	4	5	6	7	8	9
14	1	10	8	7	9	3	2	4	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

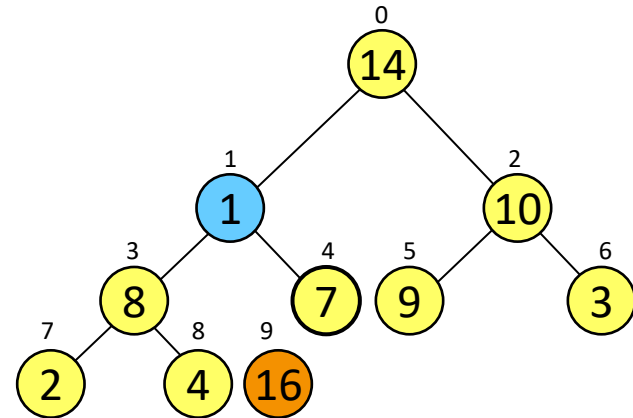


Heap Sort

0	1	2	3	4	5	6	7	8	9
14	1	10	8	7	9	3	2	4	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

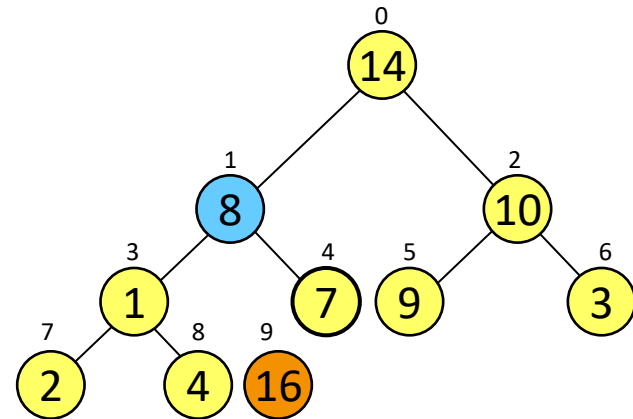


Heap Sort

0	1	2	3	4	5	6	7	8	9
14	8	10	1	7	9	3	2	4	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

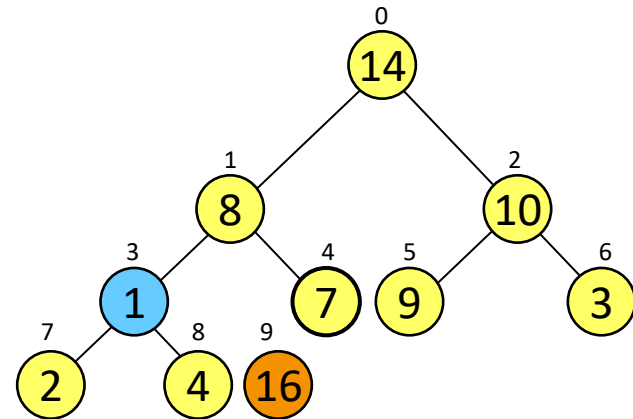


Heap Sort

0	1	2	3	4	5	6	7	8	9
14	8	10	1	7	9	3	2	4	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

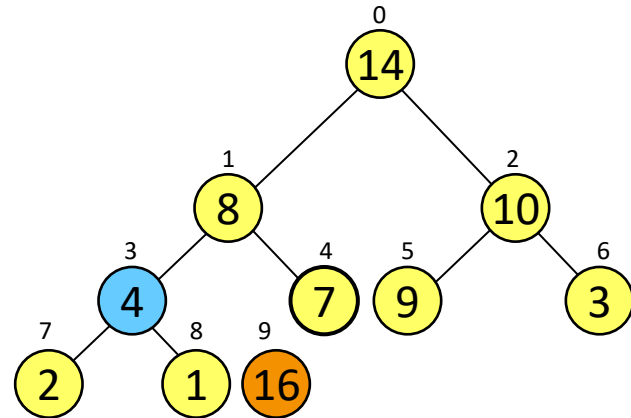


Heap Sort

0	1	2	3	4	5	6	7	8	9
14	8	10	4	7	9	3	2	1	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

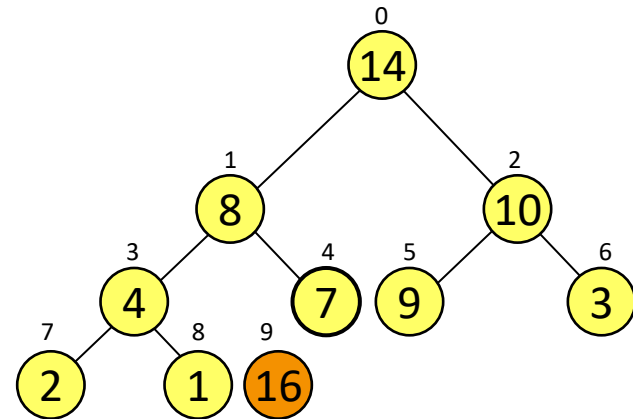


Heap Sort

0	1	2	3	4	5	6	7	8	9
14	8	10	4	7	9	3	2	1	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)

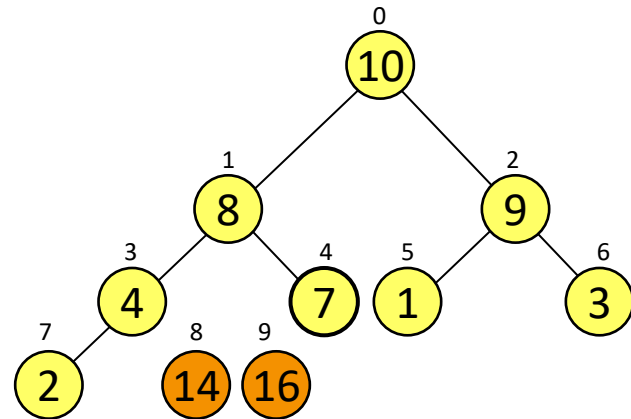


Heap Sort

0	1	2	3	4	5	6	7	8	9
10	8	9	4	7	1	3	2	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



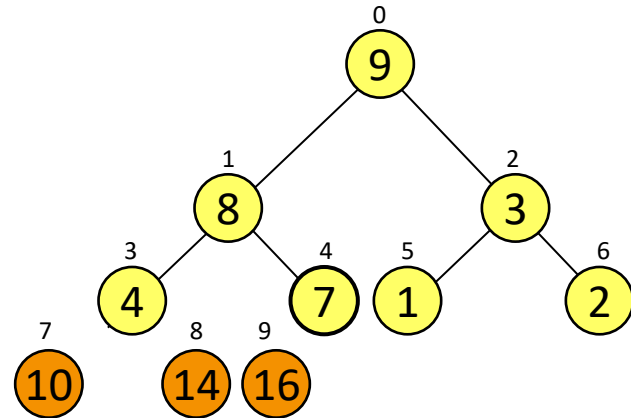
Delete 14

Heap Sort

0	1	2	3	4	5	6	7	8	9
9	8	3	4	7	1	2	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



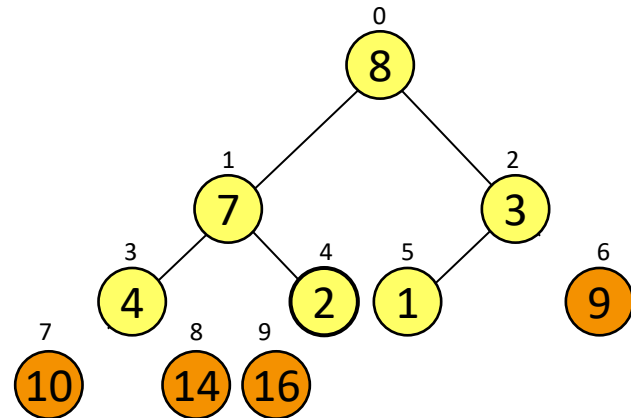
Delete 10

Heap Sort

0	1	2	3	4	5	6	7	8	9
8	7	3	4	2	1	9	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



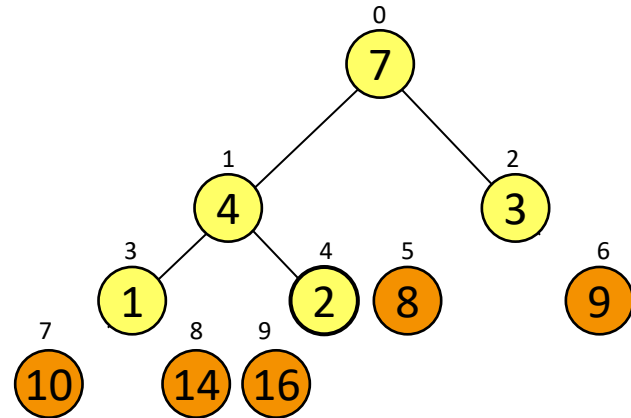
Delete 9

Heap Sort

0	1	2	3	4	5	6	7	8	9
7	4	3	1	2	8	9	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



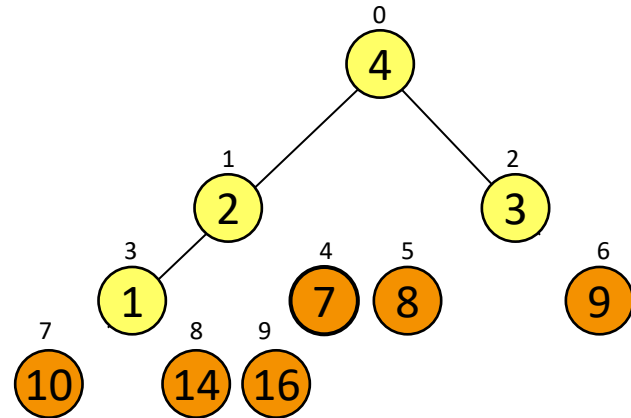
Delete 8

Heap Sort

0	1	2	3	4	5	6	7	8	9
4	2	3	1	7	8	9	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



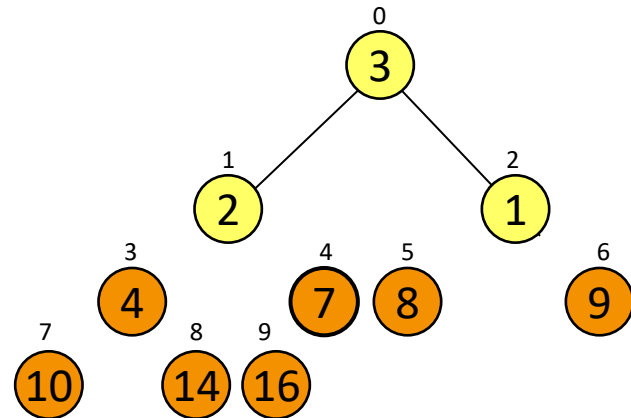
Delete 7

Heap Sort

0	1	2	3	4	5	6	7	8	9
3	2	1	4	7	8	9	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



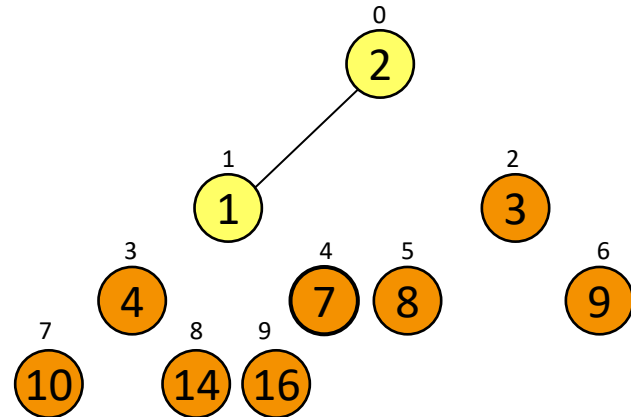
Delete 4

Heap Sort

0	1	2	3	4	5	6	7	8	9
2	1	3	4	7	8	9	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



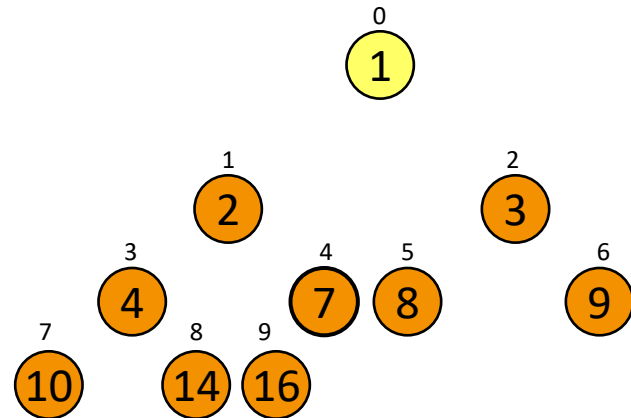
Delete 3

Heap Sort

0	1	2	3	4	5	6	7	8	9
1	2	3	4	7	8	9	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



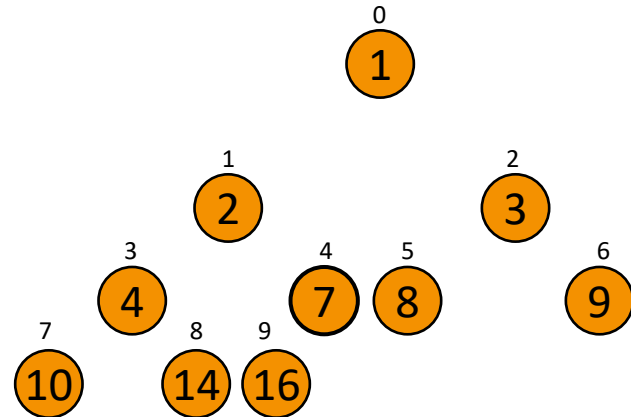
Delete 2

Heap Sort

0	1	2	3	4	5	6	7	8	9
1	2	3	4	7	8	9	10	14	16

Alg: HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown(A, 0, i-1)



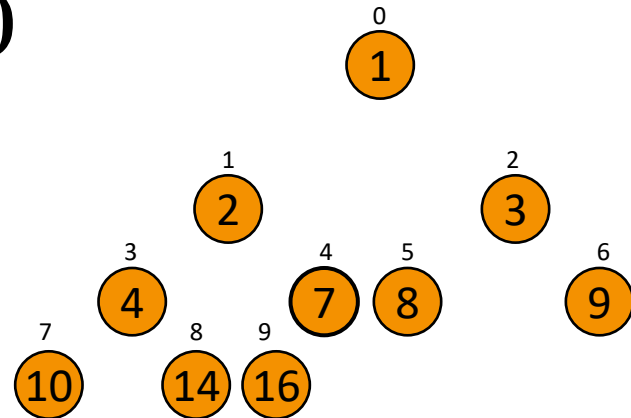
Heap Sort

0	1	2	3	4	5	6	7	8	9
1	2	3	4	7	8	9	10	14	16

Alg: HEAPSORT(A)

1. **BUILD-MAX-HEAP(A)**
2. $n = \text{length}(A) - 1$
3. for $i = n$ downto 1
4. swap $A[0]$ and $A[i]$
5. ReheapDown($A, 0, i-1$)

$O(N \log N)$



$O(N \log N)$

Time complexity: **$O(N \log N)$**